pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Vous : N DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> python -m venv env

```
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "C:\Program Files\Python311\Lib\venv\__main__.py", line 6, in <module>
    main()
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 546, in main
    builder.create(d)
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 76, in create
    self._setup_pip(context)
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 358, in _setup_pip
    self._call_new_python(context, '-m', 'ensurepip', '--upgrade',
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 354, in _call_new_python
    subprocess.check_output(args, **kwargs)
  File "C:\Program Files\Python311\Lib\subprocess.py", line 466, in check_output
    return run(*popenargs, stdout=PIPE, timeout=timeout, check=True,
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Program Files\Python311\Lib\subprocess.py", line 550, in run
    stdout, stderr = process.communicate(input, timeout=timeout)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

File "C:\Program Files\Python311\Lib\subprocess.py", line 1196, in communicate

stdout = self.stdout.read()

^^^^^^^^^^^^^^^^^^

KeyboardInterrupt

PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> ^C

PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE>

Gemini a dit

The error you are seeing (KeyboardInterrupt) happened because you manually stopped the process (likely by pressing Ctrl + C) before it could finish.

When you run python -m venv env, Python creates the virtual environment in two main steps:

1. It copies the necessary Python executable and folders.
2. It installs pip (the package manager) using a module called ensurepip.

Your trace shows that the process was interrupted exactly at step 2 (_setup_pip). Sometimes, installing pip can take a minute or two on Windows, making it look like the terminal is frozen.

How to fix it

Because the process was interrupted halfway, your env folder is likely incomplete and corrupted. Here is how to fix it:

Step 1: Delete the broken virtual environment Before trying again, delete the half-created env folder so you can start fresh. You can delete it through your File Explorer, or run this command in your terminal:

PowerShell

```
rmdir /s /q env
```

Step 2: Create the environment again Run your command one more time:

PowerShell

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
python -m venv env

Step 3: Wait patiently Give it a minute or two to finish. **Do not press** Ctrl + C, even if it looks like it is stuck. It is just quietly installing pip in the background. Once it finishes, your terminal prompt will return normally.

Would you like me to give you the command to activate the virtual environment once it successfully finishes creating?

Vous avez dit

en fr

Gemini a dit

L'erreur que vous voyez (KeyboardInterrupt) s'est produite parce que vous avez arrêté manuellement le processus (probablement en appuyant sur Ctrl + C) avant qu'il ne puisse se terminer.

Lorsque vous exécutez python -m venv env, Python crée l'environnement virtuel en deux étapes principales :

1. Il copie l'exécutable Python et les dossiers nécessaires.
2. Il installe pip (le gestionnaire de paquets) en utilisant un module appelé ensurepip.

Votre journal d'erreurs (traceback) montre que le processus a été interrompu exactement à l'étape 2 (_setup_pip). Parfois, l'installation de pip peut prendre une minute ou deux sur Windows, ce qui donne l'impression que le terminal est bloqué ou figé.

Comment corriger cela

Parce que le processus a été interrompu en cours de route, votre dossier env est probablement incomplet et corrompu. Voici comment y remédier :

Étape 1 : Supprimer l'environnement virtuel corrompu Avant de réessayer, supprimez le dossier env créé à moitié pour pouvoir repartir à zéro. Vous pouvez le supprimer via votre explorateur de fichiers, ou exécuter cette commande directement dans votre terminal :

PowerShell

rmdir /s /q env

Étape 2 : Recréer l'environnement Exécutez à nouveau votre commande initiale :

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

PowerShell

```
python -m venv env
```

Étape 3 : Patienter Laissez-lui une minute ou deux pour se terminer. **N'appuyez pas sur** Ctrl + C, même si le terminal semble bloqué. Il installe simplement pip silencieusement en arrière-plan. Une fois terminé, l'invite de votre terminal réapparaîtra normalement.

Souhaitez-vous que je vous donne la commande pour activer l'environnement virtuel une fois qu'il aura été créé avec succès ?

Vous avez dit

```
N DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> python -m venv env
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "C:\Program Files\Python311\Lib\venv\__main__.py", line 6, in <module>
    main()
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 546, in main
    builder.create(d)
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 76, in create
    self._setup_pip(context)
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 358, in _setup_pip
    self._call_new_python(context, '-m', 'ensurepip', '--upgrade',
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 354, in _call_new_python
    subprocess.check_output(args, **kwargs)
```

```
                pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
  File "C:\Program Files\Python311\Lib\subprocess.py", line 466, in check_output
    return run(*popenargs, stdout=PIPE, timeout=timeout, check=True,
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    stdout, stderr = process.communicate(input, timeout=timeout)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Program Files\Python311\Lib\subprocess.py", line 1196, in communicate
    stdout = self.stdout.read()
             ^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> ^C
Remove-Item : Impossible de trouver un paramètre positionnel acceptant l'argument « /q ».
Au caractère Ligne:1 : 1
+ rmdir /s /q env
+ ~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument : (:) [Remove-Item], ParameterBindingException
    + FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.RemoveItemCommand

PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> python -m venv env
PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> env\scripts\activate
(env) PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE>
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Gemini a dit

Super ! Vous avez réussi. Le préfixe (env) au début de votre invite de commande confirme que votre environnement virtuel est maintenant **actif** et prêt à être utilisé.

Pour information, l'erreur avec rmdir /s /q env s'est produite car vous utilisez **PowerShell**. Les arguments /s /q sont conçus pour l'invite de commande classique de Windows (cmd). Sous PowerShell, la commande correcte aurait été Remove-Item -Recurse -Force env. Cependant, la commande python -m venv env a quand même pu écraser ou finaliser l'installation sans problème !

Maintenant que vous êtes dans cet environnement isolé, la prochaine étape logique consiste à installer les paquets dont vous avez besoin pour votre projet de diagramme en barres et d'interface graphique. Vous pouvez les installer en utilisant pip :

PowerShell

```
pip install matplotlib PyQt6
```

Souhaitez-vous de l'aide pour écrire le code de votre graphique avec Matplotlib, ou pour savoir comment l'intégrer proprement dans une fenêtre PyQt6 ?

Vous avez dit

Voici mon programme Python matplotlib pour CREER DES DIAGRAMMES EN BARRE : # DIAGRAMMES EN BARRES avec PYTHON

```
import matplotlib.pyplot as plt

import numpy as np


species = ("Adelie", "Chinstrap", "Gentoo")

penguin_means = {

    'Bill Depth': (18.35, 18.43, 14.98),

    'Bill Length': (38.79, 48.83, 47.50),

    'Flipper Length': (189.95, 195.82, 217.19),
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
}


x = np.arange(len(species))  # the label locations
width = 0.25  # the width of the bars
multiplier = 0


fig, ax = plt.subplots(layout='constrained')


for attribute, measurement in penguin_means.items():
    offset = width * multiplier
    rects = ax.bar(x + offset, measurement, width, label=attribute)
    ax.bar_label(rects, padding=3)
    multiplier += 1


# Add some text for labels, title and custom x-axis tick labels, etc.
ax.set_ylabel('Length (mm)')
ax.set_title('Penguin attributes by species')
ax.set_xticks(x + width, species)
ax.legend(loc='upper left', ncols=3)
ax.set_ylim(0, 250)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
plt.show()import matplotlib.pyplot as plt
import numpy as np

species = ("Adelie", "Chinstrap", "Gentoo")
penguin_means = {
    'Bill Depth': (18.35, 18.43, 14.98),
    'Bill Length': (38.79, 48.83, 47.50),
    'Flipper Length': (189.95, 195.82, 217.19),
}

x = np.arange(len(species))  # the label locations
width = 0.25  # the width of the bars
multiplier = 0

fig, ax = plt.subplots(layout='constrained')

for attribute, measurement in penguin_means.items():
    offset = width * multiplier
    rects = ax.bar(x + offset, measurement, width, label=attribute)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    ax.bar_label(rects, padding=3)

    multiplier += 1


# Add some text for labels, title and custom x-axis tick labels, etc.

ax.set_ylabel('Length (mm)')

ax.set_title('Penguin attributes by species')

ax.set_xticks(x + width, species)

ax.legend(loc='upper left', ncols=3)

ax.set_ylim(0, 250)


plt.show()


# Fin du langage PYTHON
```

Gemini a dit

C'est un excellent script ! Il s'agit d'un exemple très propre et classique pour créer un diagramme en barres groupées (grouped bar chart) avec Matplotlib.

Avant de l'exécuter, j'aimerais vous faire part de deux petites remarques :

1. Code dupliqué Je remarque que le code a été collé **deux fois de suite** dans votre message (l'importation et la création du graphique se répètent après le premier plt.show()). Si vous l'exécutez tel quel, Python va afficher le premier graphique, attendre que vous fermiez la fenêtre, puis générer et afficher exactement le même graphique une seconde fois. Assurez-vous de ne garder qu'une seule copie du code dans votre fichier .py.

2. Prérequis d'installation Puisque vous venez tout juste de créer et d'activer votre environnement virtuel (env), n'oubliez pas d'installer les bibliothèques requises avant de lancer le script, si ce n'est pas déjà fait :

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

PowerShell

pip install matplotlib numpy

Ce qui rend ce code efficace :

- **La logique de regroupement :** L'utilisation de la variable multiplier couplée à offset (width * multiplier) est la méthode standard et élégante pour décaler les barres de chaque attribut afin qu'elles se placent côte à côte pour chaque espèce, plutôt que de se superposer.
- **L'ajout des étiquettes :** La fonction ax.bar_label(rects, padding=3) est très pratique pour afficher la valeur exacte au-dessus de chaque barre automatiquement.
- **La disposition contrainte :** layout='constrained' assure que les éléments du graphique (comme la légende ou le titre) ne débordent pas de la fenêtre.

Comme vous travaillez actuellement sur le développement d'interfaces graphiques, la fenêtre d'affichage par défaut de Matplotlib pourrait ne pas suffire pour un logiciel complet.

Souhaitez-vous que je vous montre comment adapter ce code pour intégrer directement ce graphique à l'intérieur d'une fenêtre **PyQt6** en utilisant le widget FigureCanvasQTAgg ?

Vous avez dit

N DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> python -m venv env

Traceback (most recent call last):

File "<frozen runpy>", line 198, in _run_module_as_main

File "<frozen runpy>", line 88, in _run_code

File "C:\Program Files\Python311\Lib\venv__main__.py", line 6, in <module>

main()

File "C:\Program Files\Python311\Lib\venv__init__.py", line 546, in main

builder.create(d)

File "C:\Program Files\Python311\Lib\venv__init__.py", line 76, in create

```
                pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
  self._setup_pip(context)
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 358, in _setup_pip
    self._call_new_python(context, '-m', 'ensurepip', '--upgrade',
  File "C:\Program Files\Python311\Lib\venv\__init__.py", line 354, in _call_new_python
    subprocess.check_output(args, **kwargs)
  File "C:\Program Files\Python311\Lib\subprocess.py", line 466, in check_output
    return run(*popenargs, stdout=PIPE, timeout=timeout, check=True,
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    stdout, stderr = process.communicate(input, timeout=timeout)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Program Files\Python311\Lib\subprocess.py", line 1196, in communicate
    stdout = self.stdout.read()
             ^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> ^C
Remove-Item : Impossible de trouver un paramètre positionnel acceptant l'argument « /q ».
+ ~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument : (:) [Remove-Item], ParameterBindingException
    + FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.RemoveItemCommand
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> python -m venv env

PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> env\scripts\activate

(env) PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE> pip install matplotlib PyQt6


Collecting matplotlib

  Using cached matplotlib-3.10.8-cp311-cp311-win_amd64.whl.metadata (52 kB)

Collecting PyQt6

  Downloading pyqt6-6.11.0-cp310-abi3-win_amd64.whl.metadata (2.2 kB)

Collecting contourpy>=1.0.1 (from matplotlib)

  Using cached contourpy-1.3.3-cp311-cp311-win_amd64.whl.metadata (5.5 kB)

Collecting cycler>=0.10 (from matplotlib)

  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)

Collecting fonttools>=4.22.0 (from matplotlib)

  Downloading fonttools-4.62.1-cp311-cp311-win_amd64.whl.metadata (119 kB)

     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 119.8/119.8 kB 155.8 kB/s eta 0:00:00

Collecting kiwisolver>=1.3.1 (from matplotlib)

  Using cached kiwisolver-1.5.0-cp311-cp311-win_amd64.whl.metadata (5.2 kB)

Collecting numpy>=1.23 (from matplotlib)

  Downloading numpy-2.4.4-cp311-cp311-win_amd64.whl.metadata (6.6 kB)

Collecting packaging>=20.0 (from matplotlib)
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
  Using cached packaging-26.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pillow>=8 (from matplotlib)
  Downloading pillow-12.2.0-cp311-cp311-win_amd64.whl.metadata (9.0 kB)
Collecting pyparsing>=3 (from matplotlib)
  Using cached pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Collecting python-dateutil>=2.7 (from matplotlib)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting PyQt6-sip<14,>=13.8 (from PyQt6)
  Using cached pyqt6_sip-13.11.1-cp311-cp311-win_amd64.whl.metadata (516 bytes)
Collecting PyQt6-Qt6<6.12.0,>=6.11.0 (from PyQt6)
  Downloading pyqt6_qt6-6.11.0-py3-none-win_amd64.whl.metadata (552 bytes)
Collecting six>=1.5 (from python-dateutil>=2.7->matplotlib)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Using cached matplotlib-3.10.8-cp311-cp311-win_amd64.whl (8.1 MB)
Downloading pyqt6-6.11.0-cp310-abi3-win_amd64.whl (6.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.8/6.8 MB 92.1 kB/s eta 0:00:00
Using cached contourpy-1.3.3-cp311-cp311-win_amd64.whl (225 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.62.1-cp311-cp311-win_amd64.whl (2.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 108.0 kB/s eta 0:00:00
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Using cached kiwisolver-1.5.0-cp311-cp311-win_amd64.whl (73 kB)

Downloading numpy-2.4.4-cp311-cp311-win_amd64.whl (12.6 MB)

━━━

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Requirement already satisfied: numpy in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram en barre\env\lib\site-packages (2.4.4)

Requirement already satisfied: contourpy>=1.0.1 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram en barre\env\lib\site-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram

en barre\env\lib\site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram en barre\env\lib\site-packages (from matplotlib) (4.62.1)

Requirement already satisfied: kiwisolver>=1.3.1 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram en barre\env\lib\site-packages (from matplotlib) (1.5.0)

Requirement already satisfied: packaging>=20.0 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram en barre\env\lib\site-packages (from matplotlib) (26.0)

Requirement already satisfied: pillow>=8 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram en

barre\env\lib\site-packages (from matplotlib) (12.2.0)

Requirement already satisfied: pyparsing>=3 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram

en barre\env\lib\site-packages (from matplotlib) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib

diagram en barre\env\lib\site-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in d:\python django hopes\github22\logiciel-pyqt6-cours\program python mario\matplotlib diagram en barre\env\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

[notice] A new release of pip is available: 24.0 -> 26.0.1

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

[notice] To update, run: python.exe -m pip install --upgrade pip

(env) PS D:\PYTHON DJANGO HOPES\github22\LOGICIEL-PyQt6-COURS\Program Python MARIO\matplotlib DIAGRAM EN BARRE>

Gemini a dit

C'est parfait ! Tout s'est déroulé exactement comme prévu.

Voici ce que votre terminal nous dit :

1. Vous avez installé matplotlib et PyQt6 avec succès.
2. Vous avez généré votre fichier requirements.txt (c'est une excellente pratique, surtout quand on pousse ensuite son code sur GitHub !).
3. Lorsque vous avez relancé l'installation pour matplotlib et numpy, le système a logiquement répondu Requirement already satisfied (Condition déjà satisfaite), car numpy s'était déjà installé automatiquement comme dépendance de Matplotlib.

Le petit message [notice] A new release of pip is available est juste informatif. Vous pouvez l'ignorer ou le mettre à jour plus tard avec la commande suggérée, cela ne bloquera en rien votre travail.

Puisque tout est prêt, voici comment adapter votre script sur les pingouins pour l'intégrer directement dans une véritable **interface graphique PyQt6** au lieu d'utiliser la fenêtre d'affichage basique de Matplotlib.

Code : Matplotlib intégré dans PyQt6

Créez un nouveau fichier (par exemple main.py) dans votre dossier et collez ce code :

Python

```
import sys

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
class FenetrePrincipale(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Diagramme en Barres avec PyQt6 et Matplotlib")
        self.setGeometry(100, 100, 800, 600)

        # 1. Création du widget central et de son layout (mise en page)
        widget_central = QWidget()
        self.setCentralWidget(widget_central)
        layout = QVBoxLayout(widget_central)

        # 2. Création de la figure Matplotlib et du Canvas (la toile pour PyQt6)
        # layout='constrained' garde l'affichage propre
        fig, self.ax = plt.subplots(layout='constrained')
        self.canvas = FigureCanvas(fig)

        # On ajoute le graphique au layout de la fenêtre
        layout.addWidget(self.canvas)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    # 3. Appel de la fonction pour dessiner le graphique
    self.dessiner_graphique()


def dessiner_graphique(self):
    # Vos données d'origine
    species = ("Adelie", "Chinstrap", "Gentoo")
    penguin_means = {
        'Bill Depth': (18.35, 18.43, 14.98),
        'Bill Length': (38.79, 48.83, 47.50),
        'Flipper Length': (189.95, 195.82, 217.19),
    }

    x = np.arange(len(species))
    width = 0.25
    multiplier = 0

    # La boucle d'origine modifiée pour utiliser self.ax
    for attribute, measurement in penguin_means.items():
        offset = width * multiplier
        rects = self.ax.bar(x + offset, measurement, width, label=attribute)
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        self.ax.bar_label(rects, padding=3)
        multiplier += 1

    # Configuration des labels et du titre
    self.ax.set_ylabel('Length (mm)')
    self.ax.set_title('Penguin attributes by species')
    self.ax.set_xticks(x + width, species)
    self.ax.legend(loc='upper left', ncols=3)
    self.ax.set_ylim(0, 250)

# Lancement de l'application PyQt6
if __name__ == '__main__':
    app = QApplication(sys.argv)
    fenetre = FenetrePrincipale()
    fenetre.show()
    sys.exit(app.exec())
```

Qu'est-ce qui a changé ?

- Nous avons importé FigureCanvasQTAgg. C'est le "pont" qui permet de transformer une figure Matplotlib en un widget PyQt6 que l'on peut insérer dans une fenêtre.
- Nous avons encapsulé votre logique de création de graphique dans une méthode dessiner_graphique() à l'intérieur d'une classe QMainWindow.

---

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Souhaitez-vous exécuter ce code de votre côté pour voir le résultat, ou voulez-vous que je vous explique comment ajouter des boutons (par exemple pour changer les données du graphique) en dessous de ce diagramme ?

Vous avez dit

Vous pouvez ajouter des boutons (par exemple pour changer les données du graphique) en dessous de ce diagramme

Gemini a dit

C'est une excellente idée ! Rendre un graphique interactif est tout l'intérêt d'utiliser une interface graphique comme PyQt6.

Pour accomplir cela, nous allons utiliser deux concepts clés :

1. **Les Layouts (Mises en page) :** Nous allons créer une boîte verticale (QVBoxLayout) qui contiendra le graphique en haut, et une boîte horizontale (QHBoxLayout) en bas pour aligner nos boutons côte à côte.
2. **Le rafraîchissement (Redraw) :** Lorsque vous cliquez sur un bouton, le programme doit effacer l'ancien graphique (self.ax.clear()), dessiner les nouvelles données, puis dire à PyQt6 de repeindre la toile (self.canvas.draw()).

Voici le code complet mis à jour. Copiez-le et exécutez-le à la place du précédent :

Python

```
import sys

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QHBoxLayout, QWidget, QPushButton


class FenetrePrincipale(QMainWindow):

    def __init__(self):
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
super().__init__()

self.setWindowTitle("Diagramme Interactif avec PyQt6")

self.setGeometry(100, 100, 800, 650)


# 1. Widget central et Layout Principal (Vertical)

widget_central = QWidget()

self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


# 2. Zone du Graphique Matplotlib

fig, self.ax = plt.subplots(layout='constrained')

self.canvas = FigureCanvas(fig)

layout_principal.addWidget(self.canvas)


# 3. Zone des Boutons (Layout Horizontal)

layout_boutons = QHBoxLayout()


# Bouton 1 : Pingouins

self.btn_pingouins = QPushButton("Afficher les Pingouins")

self.btn_pingouins.clicked.connect(self.charger_donnees_pingouins) # Connexion au clic
```

```
    # Bouton 2 : Renards (Nouvelles données d'exemple)
    self.btn_renards = QPushButton("Afficher les Renards")
    self.btn_renards.clicked.connect(self.charger_donnees_renards) # Connexion au clic

    # Ajout des boutons dans leur layout horizontal
    layout_boutons.addWidget(self.btn_pingouins)
    layout_boutons.addWidget(self.btn_renards)

    # Ajout de la zone des boutons en bas du layout principal
    layout_principal.addLayout(layout_boutons)

    # 4. Affichage initial (au lancement, on montre les pingouins)
    self.charger_donnees_pingouins()

# --- Méthodes pour préparer les données ---

def charger_donnees_pingouins(self):
    categories = ("Adelie", "Chinstrap", "Gentoo")
    donnees = {
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        'Bill Depth': (18.35, 18.43, 14.98),
        'Bill Length': (38.79, 48.83, 47.50),
        'Flipper Length': (189.95, 195.82, 217.19),
    }
    self.dessiner_graphique("Attributs des Pingouins (mm)", categories, donnees)

def charger_donnees_renards(self):
    # Nouvelles données inventées pour l'exemple
    categories = ("Polaire", "Roux", "Fennec")
    donnees = {
        'Taille moyenne': (55.0, 65.5, 38.0),
        'Poids moyen': (4.5, 6.0, 1.2),
        'Vitesse max': (45.0, 50.0, 32.0),
    }
    self.dessiner_graphique("Attributs des Renards", categories, donnees)

# --- Méthode générique pour dessiner le graphique ---

def dessiner_graphique(self, titre, categories, donnees):
    # A. ÉTAPE CRUCIALE : Effacer le graphique précédent
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
self.ax.clear()


# B. Préparer les nouvelles données
x = np.arange(len(categories))
width = 0.25
multiplier = 0


# C. Dessiner les nouvelles barres
for attribute, measurement in donnees.items():
    offset = width * multiplier
    rects = self.ax.bar(x + offset, measurement, width, label=attribute)
    self.ax.bar_label(rects, padding=3)
    multiplier += 1


# D. Configurer les labels et le titre
self.ax.set_ylabel('Mesures')
self.ax.set_title(titre)
self.ax.set_xticks(x + width, categories)
self.ax.legend(loc='upper left', ncols=3)
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

```
    # Calculer le maximum des données pour ajuster l'axe Y automatiquement

    max_val = max([max(v) for v in donnees.values()])

    self.ax.set_ylim(0, max_val + (max_val * 0.2)) # On ajoute 20% d'espace en haut pour que les labels respirent


    # E. ÉTAPE CRUCIALE : Dire à l'interface de se rafraîchir

    self.canvas.draw()



if __name__ == '__main__':

  app = QApplication(sys.argv)

  fenetre = FenetrePrincipale()

  fenetre.show()

  sys.exit(app.exec())
```

Ce qui est important à retenir ici :

- self.ax.clear() : Sans cette ligne, à chaque fois que vous cliqueriez sur un bouton, Matplotlib dessinerait les nouvelles barres *par-dessus* les anciennes, ce qui créerait un gros gribouillis illisible.
- self.canvas.draw() : Sans cette ligne, Matplotlib mettrait bien à jour le graphique en mémoire, mais l'interface visuelle PyQt6 ne le saurait pas, et votre écran ne changerait pas.
- max_val : J'ai ajouté un petit calcul mathématique automatique pour la limite Y (self.ax.set_ylim). Ainsi, peu importe si vos données vont de 0 à 50 (comme les renards) ou de 0 à 250 (comme les pingouins), le graphique s'ajustera parfaitement sans couper les barres.

---

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Voulez-vous que l'on applique ce même principe d'interface avec des boutons pour d'autres projets sur lesquels vous travaillez, comme l'affichage de statistiques pour votre logiciel de gestion de location de voitures ?

Vous avez dit

Pouvez vous mettre des unités de mesure dans le diagramme en barre

Gemini a dit

C'est une excellente idée ! L'ajout d'unités de mesure est essentiel, en particulier pour nos "Renards" où l'on mélange des centimètres, des kilos et des kilomètres par heure.

Pour faire cela dans Matplotlib, nous ne pouvons plus utiliser le comportement par défaut de ax.bar_label. Nous allons plutôt lui fournir une liste de **textes personnalisés** pour chaque barre, en combinant la valeur numérique avec son unité.

Voici comment modifier notre code pour intégrer une unité spécifique à chaque catégorie.

Le code mis à jour avec les unités de mesure

Vous pouvez remplacer tout le code de votre fichier main.py par celui-ci :

Python

```
import sys

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QHBoxLayout, QWidget, QPushButton


class FenetrePrincipale(QMainWindow):

    def __init__(self):
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
super().__init__()

self.setWindowTitle("Diagramme Interactif avec PyQt6 et Unités")

self.setGeometry(100, 100, 900, 650) # Fenêtre un peu plus large pour le texte


# 1. Widget central et Layout Principal

widget_central = QWidget()

self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


# 2. Zone du Graphique

fig, self.ax = plt.subplots(layout='constrained')

self.canvas = FigureCanvas(fig)

layout_principal.addWidget(self.canvas)


# 3. Zone des Boutons

layout_boutons = QHBoxLayout()


self.btn_pingouins = QPushButton("Afficher les Pingouins")

self.btn_pingouins.clicked.connect(self.charger_donnees_pingouins)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    self.btn_renards = QPushButton("Afficher les Renards")
    self.btn_renards.clicked.connect(self.charger_donnees_renards)

    layout_boutons.addWidget(self.btn_pingouins)
    layout_boutons.addWidget(self.btn_renards)
    layout_principal.addLayout(layout_boutons)

    # 4. Affichage initial
    self.charger_donnees_pingouins()

# --- NOUVEAU : On ajoute un dictionnaire pour les unités ---

def charger_donnees_pingouins(self):
    categories = ("Adelie", "Chinstrap", "Gentoo")
    donnees = {
        'Bill Depth': (18.35, 18.43, 14.98),
        'Bill Length': (38.79, 48.83, 47.50),
        'Flipper Length': (189.95, 195.82, 217.19),
    }
    # Toutes les mesures des pingouins sont en millimètres
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
    unites = {
        'Bill Depth': 'mm',
        'Bill Length': 'mm',
        'Flipper Length': 'mm'
    }
    self.dessiner_graphique("Attributs des Pingouins", categories, donnees, unites)

def charger_donnees_renards(self):
    categories = ("Polaire", "Roux", "Fennec")
    donnees = {
        'Taille moyenne': (55.0, 65.5, 38.0),
        'Poids moyen': (4.5, 6.0, 1.2),
        'Vitesse max': (45.0, 50.0, 32.0),
    }
    # Les mesures des renards ont des unités différentes !
    unites = {
        'Taille moyenne': 'cm',
        'Poids moyen': 'kg',
        'Vitesse max': 'km/h'
    }
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    self.dessiner_graphique("Attributs des Renards", categories, donnees, unites)


# --- MISE À JOUR : La méthode prend maintenant les unités en paramètre ---


def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()


    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0


    for attribute, measurement in donnees.items():
        offset = width * multiplier
        rects = self.ax.bar(x + offset, measurement, width, label=attribute)


        # --- L'ASTUCE EST ICI ---
        # On récupère l'unité correspondant à l'attribut en cours
        unite = unites[attribute]


        # On crée une liste de textes : ex ["55.0 cm", "65.5 cm", "38.0 cm"]
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            etiquettes = [f"{val} {unite}" for val in measurement]

            # On passe cette liste à matplotlib (fontsize réduit un peu pour que ça rentre bien)
            self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)

            multiplier += 1

        self.ax.set_title(titre)
        self.ax.set_xticks(x + width, categories)
        self.ax.legend(loc='upper left', ncols=3)

        # On ajuste l'axe Y pour laisser de la place aux textes plus longs
        max_val = max([max(v) for v in donnees.values()])
        self.ax.set_ylim(0, max_val + (max_val * 0.25)) # 25% d'espace en haut

        self.canvas.draw()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    fenetre = FenetrePrincipale()
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
fenetre.show()

sys.exit(app.exec())
```

Ce qui a changé en détail :

- **Le dictionnaire** unites **:** Dans chaque fonction de chargement, nous déclarons à quelle unité correspond chaque attribut.
- **La création des** etiquettes **:** Au lieu de laisser Matplotlib écrire juste 55.0, la ligne [f"{val} {unite}" for val in measurement] utilise les *f-strings* de Python pour accrocher l'unité à la valeur.
- **Le paramètre** labels= **:** On force ax.bar_label à utiliser notre liste de textes sur mesure au lieu de deviner tout seul. J'ai aussi ajouté fontsize=9 pour éviter que les étiquettes ne se chevauchent avec les nouvelles lettres ajoutées.

Souhaitez-vous que nous essayions d'adapter ce code pour afficher les statistiques d'un vrai cas pratique, comme par exemple le nombre de "Jours loués" et les "Revenus en €" pour différents véhicules dans votre logiciel de location ?

Vous avez dit

Et si l'on suppose qu'il y a beaucoup d'animaux autres que le renards et les pingouins ? Comment feriez vous ?

Gemini a dit

C'est une excellente question de conception ! C'est exactement le moment où l'on passe d'un "petit script de test" à une "véritable architecture logicielle".

Si vous avez des dizaines ou des centaines d'animaux, créer un bouton et une fonction pour *chaque* animal est impossible à gérer. L'interface serait illisible et le code beaucoup trop répétitif.

Pour résoudre ce problème, il faut changer deux choses :

1. **L'Interface (PyQt6) :** Remplacer les boutons par un **menu déroulant** (QComboBox) ou une liste.
2. **La Structure des données (Python) :** Créer un dictionnaire global (ou lire un fichier JSON/CSV) qui contient tous les animaux, et créer une **seule fonction générique** pour les charger.

Voici comment transformer notre code pour qu'il puisse gérer une infinité d'animaux de manière très élégante :

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Le code optimisé avec un Menu Déroulant (QComboBox)

Python

```
import sys

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QComboBox, QLabel, QHBoxLayout


class FenetrePrincipale(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Base de données Animale avec PyQt6")

        self.setGeometry(100, 100, 900, 650)


        # --- 1. NOTRE "BASE DE DONNÉES" CENTRALE ---

        # Si vous aviez des milliers d'animaux, vous liriez ceci depuis un fichier .json ou une base SQL (comme avec Django !)

        self.base_animaux = {

            "Pingouins": {

                "categories": ("Adelie", "Chinstrap", "Gentoo"),

                "donnees": {
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py


```
        'Profondeur du bec': (18.35, 18.43, 14.98),
        'Longueur du bec': (38.79, 48.83, 47.50),
        'Longueur des nageoires': (189.95, 195.82, 217.19),
    },
    "unites": {'Profondeur du bec': 'mm', 'Longueur du bec': 'mm', 'Longueur des nageoires': 'mm'}
},
"Renards": {
    "categories": ("Polaire", "Roux", "Fennec"),
    "donnees": {
        'Taille moyenne': (55.0, 65.5, 38.0),
        'Poids moyen': (4.5, 6.0, 1.2),
        'Vitesse max': (45.0, 50.0, 32.0),
    },
    "unites": {'Taille moyenne': 'cm', 'Poids moyen': 'kg', 'Vitesse max': 'km/h'}
},
"Félins": {
    "categories": ("Lion", "Tigre", "Guépard"),
    "donnees": {
        'Poids': (190.0, 220.0, 50.0),
        'Vitesse max': (80.0, 65.0, 120.0),
```

```
        pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
      },
      "unites": {'Poids': 'kg', 'Vitesse max': 'km/h'}
    }
}

# --- 2. INTERFACE GRAPHIQUE ---
widget_central = QWidget()
self.setCentralWidget(widget_central)
layout_principal = QVBoxLayout(widget_central)

# Création du menu déroulant (QComboBox)
layout_menu = QHBoxLayout()
label_menu = QLabel("Choisissez une espèce :")
self.menu_deroulant = QComboBox()

# On remplit le menu avec les noms de nos animaux (les clés de notre dictionnaire)
self.menu_deroulant.addItems(self.base_animaux.keys())

# On connecte le changement de choix à notre fonction de mise à jour
self.menu_deroulant.currentTextChanged.connect(self.changer_animal)
```

```
    layout_menu.addWidget(label_menu)
    layout_menu.addWidget(self.menu_deroulant)
    layout_menu.addStretch() # Pousse le menu sur la gauche

    layout_principal.addLayout(layout_menu)

    # Création de la zone graphique
    fig, self.ax = plt.subplots(layout='constrained')
    self.canvas = FigureCanvas(fig)
    layout_principal.addWidget(self.canvas)

    # --- 3. AFFICHAGE INITIAL ---
    # On simule un premier clic avec la valeur par défaut du menu
    self.changer_animal(self.menu_deroulant.currentText())


# --- 4. LA FONCTION MAGIQUE QUI GÈRE TOUT ---
def changer_animal(self, nom_espece):
    # On récupère toutes les infos de l'espèce choisie depuis notre dictionnaire
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    infos = self.base_animaux[nom_espece]

    titre = f"Comparaison des {nom_espece}"
    # On envoie ces infos à la fonction qui dessine
    self.dessiner_graphique(titre, infos["categories"], infos["donnees"], infos["unites"])


def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()

    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0

    for attribute, measurement in donnees.items():
        offset = width * multiplier
        rects = self.ax.bar(x + offset, measurement, width, label=attribute)

        unite = unites[attribute]
        etiquettes = [f"{val} {unite}" for val in measurement]
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)
            multiplier += 1

        self.ax.set_title(titre)
        self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories) # Centrage dynamique des labels X
        self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1)) # Sort la légende du graphique si elle est trop grosse

        max_val = max([max(v) for v in donnees.values()])
        self.ax.set_ylim(0, max_val + (max_val * 0.25))

        self.canvas.draw()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    fenetre = FenetrePrincipale()
    fenetre.show()
    sys.exit(app.exec())
```

Les nouveautés introduites :

- **Le dictionnaire universel (**self.base_animaux**) :** Regardez comment les données sont stockées au début. Si vous vouliez ajouter 50 animaux de plus, vous n'auriez **aucune ligne de code à changer dans les fonctions**. Il suffirait d'ajouter des lignes dans ce dictionnaire.

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

- **Le** QComboBox **:** C'est le widget classique pour faire des listes déroulantes en PyQt6. La méthode .addItems() permet de le remplir d'un seul coup.
- **Le centrage dynamique (**set_xticks**) :** J'ai légèrement modifié la formule mathématique du centrage des textes sous les barres (x + (width * (len(donnees) - 1) / 2)). Ainsi, que l'animal ait 2 caractéristiques (Félins) ou 3 (Pingouins), le texte de l'espèce restera parfaitement centré sous son groupe de barres.

Cette méthode (séparer complètement l'interface graphique de la base de données) est la base pour développer des logiciels robustes.

Puisque vous travaillez sur un logiciel de gestion de location de voitures avec PyQt6, voudriez-vous que l'on transpose cette exact même architecture pour afficher les statistiques de votre flotte (par exemple : choisir un modèle de voiture dans le menu déroulant pour voir ses jours loués ou ses revenus) ?

Vous avez dit

Et si l'on voulait obtenir toutes les données à partir d'un FICHIER JSON nommé matplotlib_gemini3_1_pro4.json de sorte q'AUCUNE DONNEES NE SE TROUVE PLUS DANS LE FICHIER matplotlib_gemini3_1_pro4.py ?

Gemini a dit

C'est la pratique de développement ultime ! Séparer complètement les données (le "Modèle") de l'interface visuelle (la "Vue") est au cœur de l'architecture logicielle professionnelle.

Pour y parvenir, nous avons besoin de la bibliothèque standard json de Python. L'avantage du JSON est qu'il est très similaire aux dictionnaires Python, mais avec quelques règles strictes (par exemple, utiliser uniquement des guillemets doubles "" et des crochets [] pour les listes).

Voici comment diviser votre projet en deux fichiers distincts. Assurez-vous de placer ces deux fichiers dans le **même dossier**.

Fichier 1 : matplotlib_gemini3_1_pro4.json

Créez ce fichier et collez-y les données pures. Il n'y a plus aucune trace de code Python ici.

JSON

```
{
  "Pingouins": {
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py


```
    "categories": ["Adelie", "Chinstrap", "Gentoo"],
    "donnees": {
        "Profondeur du bec": [18.35, 18.43, 14.98],
        "Longueur du bec": [38.79, 48.83, 47.50],
        "Longueur des nageoires": [189.95, 195.82, 217.19]
    },
    "unites": {
        "Profondeur du bec": "mm",
        "Longueur du bec": "mm",
        "Longueur des nageoires": "mm"
    }
},
"Renards": {
    "categories": ["Polaire", "Roux", "Fennec"],
    "donnees": {
        "Taille moyenne": [55.0, 65.5, 38.0],
        "Poids moyen": [4.5, 6.0, 1.2],
        "Vitesse max": [45.0, 50.0, 32.0]
    },
    "unites": {
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py


```
        "Taille moyenne": "cm",
        "Poids moyen": "kg",
        "Vitesse max": "km/h"
      }
    },
    "Félins": {
      "categories": ["Lion", "Tigre", "Guépard"],
      "donnees": {
        "Poids": [190.0, 220.0, 50.0],
        "Vitesse max": [80.0, 65.0, 120.0]
      },
      "unites": {
        "Poids": "kg",
        "Vitesse max": "km/h"
      }
    }
}
```


Fichier 2 : matplotlib_gemini3_1_pro4.py

Ce fichier est maintenant beaucoup plus court et propre. Il se contente de lire le fichier JSON et de construire l'interface graphique. J'ai ajouté une boîte de dialogue (QMessageBox) pour afficher une erreur si le fichier JSON est introuvable (une excellente pratique en PyQt6).

Python

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
import sys

import json # Importation du module JSON

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QComboBox, QLabel, QHBoxLayout, QMessageBox


class FenetrePrincipale(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Base de données Animale depuis JSON")

        self.setGeometry(100, 100, 900, 650)


        # --- 1. CHARGEMENT DES DONNÉES DEPUIS LE FICHIER JSON ---

        self.base_animaux = self.charger_donnees_json("matplotlib_gemini3_1_pro4.json")


        # Si le fichier n'a pas pu être chargé, on arrête l'initialisation de l'interface

        if not self.base_animaux:

            return
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
# --- 2. INTERFACE GRAPHIQUE ---

widget_central = QWidget()

self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


# Création du menu déroulant

layout_menu = QHBoxLayout()

label_menu = QLabel("Choisissez une espèce :")

self.menu_deroulant = QComboBox()


# On remplit le menu avec les clés du JSON

self.menu_deroulant.addItems(self.base_animaux.keys())

self.menu_deroulant.currentTextChanged.connect(self.changer_animal)


layout_menu.addWidget(label_menu)

layout_menu.addWidget(self.menu_deroulant)

layout_menu.addStretch()

layout_principal.addLayout(layout_menu)


# Création de la zone graphique
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
    fig, self.ax = plt.subplots(layout='constrained')

    self.canvas = FigureCanvas(fig)

    layout_principal.addWidget(self.canvas)


    # Affichage initial

    self.changer_animal(self.menu_deroulant.currentText())


# --- MÉTHODE DE LECTURE DU FICHIER JSON ---

def charger_donnees_json(self, chemin_fichier):

    try:

        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:

            donnees = json.load(fichier)

            return donnees

    except FileNotFoundError:

        QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable. Assurez-vous qu'il est dans le même dossier que le script.")

        return {}

    except json.JSONDecodeError:

        QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON.")

        return {}
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
def changer_animal(self, nom_espece):
    infos = self.base_animaux[nom_espece]
    titre = f"Comparaison des {nom_espece}"
    self.dessiner_graphique(titre, infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()

    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0

    for attribute, measurement in donnees.items():
        offset = width * multiplier
        rects = self.ax.bar(x + offset, measurement, width, label=attribute)

        unite = unites[attribute]
        etiquettes = [f"{val} {unite}" for val in measurement]

        self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        multiplier += 1


        self.ax.set_title(titre)

        self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)

        self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))


        max_val = max([max(v) for v in donnees.values()])

        self.ax.set_ylim(0, max_val + (max_val * 0.25))


        self.canvas.draw()


if __name__ == '__main__':

    app = QApplication(sys.argv)

    fenetre = FenetrePrincipale()

    fenetre.show()

    sys.exit(app.exec())
```

La magie de cette approche

Maintenant, vous pouvez envoyer le fichier .exe ou .py à un utilisateur, et s'il veut ajouter un nouvel animal (ou modifier une valeur erronée), il lui suffit d'ouvrir le fichier .json avec le Bloc-notes. Le programme lira les nouvelles données au prochain lancement sans avoir besoin d'être recompilé ou modifié.

---

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Voudriez-vous appliquer cette logique de fichier externe pour stocker les données d'autres projets, comme le catalogue de véhicules de votre logiciel de gestion de location ?

Vous avez dit

Juste une question : Si le fichier JSON est syntaxiquement incorrect alors un message devrait normalement apparaitre disant que le fichier JSON contient une erreur de syntaxe

Gemini a dit

Vous avez l'œil ! Et vous avez tout à fait raison : une application robuste ne doit pas planter silencieusement ou cracher du texte rouge incompréhensible dans le terminal si le fichier externe a été mal modifié par un utilisateur.

C'est d'ailleurs exactement pour cela que j'ai déjà intégré cette sécurité dans le code Python que je vous ai fourni à l'étape précédente.

Si vous regardez la méthode charger_donnees_json, vous y trouverez ce bloc de code précis :

Python

```
except json.JSONDecodeError:
    QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON.")
    return {}
```

Faites le test vous-même !

Pour voir cette sécurité en action, je vous invite à provoquer l'erreur volontairement :

1. Ouvrez votre fichier matplotlib_gemini3_1_pro4.json.
2. Supprimez **une seule virgule** (par exemple, celle à la fin de la ligne "Profondeur du bec": [18.35, 18.43, 14.98]).
3. Enregistrez le fichier JSON.
4. Lancez votre script Python.

Au lieu de voir le programme planter dans le terminal PowerShell, PyQt6 va intercepter l'erreur (json.JSONDecodeError) et faire apparaître une belle fenêtre contextuelle (un QMessageBox) avec une croix rouge, indiquant : *"Le fichier matplotlib_gemini3_1_pro4.json contient des erreurs de syntaxe JSON."*

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

C'est la grande force de la gestion des exceptions (try... except...) en Python couplée aux boîtes de dialogue de PyQt6.

Voulez-vous que je vous montre comment ajouter un autre niveau de sécurité qui vérifierait non seulement la syntaxe, mais aussi si les clés attendues (comme "categories" ou "donnees") sont bien présentes dans le JSON avant d'essayer de dessiner le graphique ?

Vous avez dit

Oui

Gemini a dit

C'est une excellente initiative. Vérifier que les données ont la bonne *structure* (ce qu'on appelle la validation de schéma) est la marque d'un logiciel vraiment professionnel. Sans cela, si quelqu'un oublie de mettre la section "unites" pour un nouvel animal dans le fichier JSON, votre programme plantera plus tard avec un KeyError.

Pour mettre cela en place, nous allons ajouter une nouvelle méthode appelée valider_structure et modifier légèrement charger_donnees_json pour l'utiliser.

Voici les deux fonctions mises à jour. Vous pouvez simplement remplacer la fonction charger_donnees_json de votre fichier matplotlib_gemini3_1_pro4.py par ce bloc de code :

Python

```
# --- MÉTHODE DE LECTURE DU FICHIER JSON ---
def charger_donnees_json(self, chemin_fichier):
    try:
        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:
            donnees_brutes = json.load(fichier)

            # NOUVEAU : On vérifie que la structure est correcte avant de l'accepter
            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        # S'il y a des erreurs, on avertit l'utilisateur, mais on garde les données valides
        if liste_erreurs:
            message = "Certaines données du fichier JSON ont été ignorées car elles sont incomplètes :\n\n"
            message += "\n".join(liste_erreurs)
            QMessageBox.warning(self, "Avertissement de Structure", message)

        return donnees_valides

    except FileNotFoundError:
        QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable.")
        return {}
    except json.JSONDecodeError:
        QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON (ex: virgule manquante).")
        return {}

# --- NOUVELLE MÉTHODE : VALIDATION DE LA STRUCTURE ---
def valider_structure(self, donnees):
    cles_requises = ["categories", "donnees", "unites"]
    donnees_valides = {}
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
    erreurs = []


    # On parcourt chaque animal dans le JSON
    for animal, contenu in donnees.items():
        # On s'assure que le contenu est bien un dictionnaire (un objet JSON)
        if not isinstance(contenu, dict):
            erreurs.append(f"- L'entrée '{animal}' n'est pas formatée correctement.")
            continue


        # On cherche quelles clés manquent parmi celles qui sont requises
        cles_manquantes = [cle for cle in cles_requises if cle not in contenu]


        if cles_manquantes:
            erreurs.append(f"- '{animal}' : il manque les clés ({', '.join(cles_manquantes)})")
        else:
            # Si tout va bien, on ajoute cet animal aux données valides
            donnees_valides[animal] = contenu


    return donnees_valides, erreurs
```

Comment ça fonctionne en pratique :

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

1. **La validation :** La méthode valider_structure prend chaque animal un par un. Elle regarde s'il possède bien les clés "categories", "donnees", et "unites".
2. **Le tri :** Si un animal (disons "Oiseaux") n'a pas de section "unites", la fonction note l'erreur, mais ne bloque pas tout ! Elle garde les "Pingouins" et les "Renards" qui sont corrects.
3. **Le signalement :** Au retour dans charger_donnees_json, si des erreurs ont été notées, une fenêtre QMessageBox.warning s'affiche pour dire à l'utilisateur exactement ce qui manque et pour quel animal, mais le logiciel s'ouvre quand même avec les données valides.

Vous avez maintenant un script Python blindé contre les erreurs de manipulation des utilisateurs sur le fichier de données !

Maintenant que vous avez un code d'interface graphique très propre et séparé de ses données, aimeriez-vous que je vous montre comment empaqueter ce script Python (et son fichier JSON) dans un seul fichier exécutable .exe pour que vous puissiez le partager facilement ?

Vous avez dit

Pouvez vous donner le code complet ? svp

Gemini a dit

Bien sûr ! Voici la version complète et finale de votre projet, séparée en deux fichiers distincts.

Pour que cela fonctionne parfaitement, assurez-vous que les deux fichiers sont enregistrés dans le **même dossier**.

Fichier 1 : matplotlib_gemini3_1_pro4.json

Ce fichier contient uniquement vos données, structurées de manière stricte.

JSON

```
{
  "Pingouins": {
    "categories": ["Adelie", "Chinstrap", "Gentoo"],
    "donnees": {
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py


```
      "Profondeur du bec": [18.35, 18.43, 14.98],
      "Longueur du bec": [38.79, 48.83, 47.50],
      "Longueur des nageoires": [189.95, 195.82, 217.19]
    },
    "unites": {
      "Profondeur du bec": "mm",
      "Longueur du bec": "mm",
      "Longueur des nageoires": "mm"
    }
  },
  "Renards": {
    "categories": ["Polaire", "Roux", "Fennec"],
    "donnees": {
      "Taille moyenne": [55.0, 65.5, 38.0],
      "Poids moyen": [4.5, 6.0, 1.2],
      "Vitesse max": [45.0, 50.0, 32.0]
    },
    "unites": {
      "Taille moyenne": "cm",
      "Poids moyen": "kg",
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py


```
        "Vitesse max": "km/h"
      }
    },
    "Félins": {
      "categories": ["Lion", "Tigre", "Guépard"],
      "donnees": {
        "Poids": [190.0, 220.0, 50.0],
        "Vitesse max": [80.0, 65.0, 120.0]
      },
      "unites": {
        "Poids": "kg",
        "Vitesse max": "km/h"
      }
    }
}
```

Fichier 2 : matplotlib_gemini3_1_pro4.py

Ce fichier est le cœur de votre application. Il intègre PyQt6, Matplotlib, la lecture du JSON, et nos deux niveaux de sécurité (erreurs de syntaxe et validation de la structure).

Python

```
import sys
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
import json

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QComboBox, QLabel, QHBoxLayout, QMessageBox


class FenetrePrincipale(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Base de données Animale depuis JSON")

        self.setGeometry(100, 100, 900, 650)


        # --- 1. CHARGEMENT ET VALIDATION DES DONNÉES ---

        self.base_animaux = self.charger_donnees_json("matplotlib_gemini3_1_pro4.json")


        if not self.base_animaux:

            return # Arrête l'initialisation de l'interface si aucune donnée valide n'est trouvée


        # --- 2. INTERFACE GRAPHIQUE ---

        widget_central = QWidget()
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


# Menu déroulant

layout_menu = QHBoxLayout()

label_menu = QLabel("Choisissez une espèce :")

self.menu_deroulant = QComboBox()


self.menu_deroulant.addItems(self.base_animaux.keys())

self.menu_deroulant.currentTextChanged.connect(self.changer_animal)


layout_menu.addWidget(label_menu)

layout_menu.addWidget(self.menu_deroulant)

layout_menu.addStretch()

layout_principal.addLayout(layout_menu)


# Zone graphique Matplotlib

fig, self.ax = plt.subplots(layout='constrained')

self.canvas = FigureCanvas(fig)

layout_principal.addWidget(self.canvas)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    # Affichage initial
    self.changer_animal(self.menu_deroulant.currentText())

# --- 3. MÉTHODES DE LECTURE ET VALIDATION DU JSON ---
def charger_donnees_json(self, chemin_fichier):
    try:
        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:
            donnees_brutes = json.load(fichier)

            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)

            if liste_erreurs:
                message = "Certaines données du fichier JSON ont été ignorées car elles sont incomplètes :\n\n"
                message += "\n".join(liste_erreurs)
                QMessageBox.warning(self, "Avertissement de Structure", message)

            return donnees_valides

    except FileNotFoundError:
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable. Assurez-vous qu'il est dans le même dossier que le script.")
        return {}
    except json.JSONDecodeError:
        QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON (ex: virgule manquante).")
        return {}

def valider_structure(self, donnees):
    cles_requises = ["categories", "donnees", "unites"]
    donnees_valides = {}
    erreurs = []

    for animal, contenu in donnees.items():
        if not isinstance(contenu, dict):
            erreurs.append(f"- L'entrée '{animal}' n'est pas formatée correctement.")
            continue

        cles_manquantes = [cle for cle in cles_requises if cle not in contenu]

        if cles_manquantes:
            erreurs.append(f"- '{animal}' : il manque les clés ({', '.join(cles_manquantes)})")
```

```
            pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        else:
            donnees_valides[animal] = contenu

    return donnees_valides, erreurs

# --- 4. MÉTHODES DE MISE À JOUR ET DESSIN ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
        titre = f"Comparaison des {nom_espece}"
        self.dessiner_graphique(titre, infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()

    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0

    for attribute, measurement in donnees.items():
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
    offset = width * multiplier

    rects = self.ax.bar(x + offset, measurement, width, label=attribute)


    # Utilisation de .get() pour éviter un plantage si l'unité est manquante de façon inattendue

    unite = unites.get(attribute, "")

    etiquettes = [f"{val} {unite}" for val in measurement]


    self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)

    multiplier += 1


self.ax.set_title(titre)


# Centrage dynamique des labels X en fonction du nombre de séries de données

if len(donnees) > 0:

    self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)


self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))


# Ajustement automatique de l'axe Y

try:
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        max_val = max([max(v) for v in donnees.values() if v])

        self.ax.set_ylim(0, max_val + (max_val * 0.25))

    except ValueError:

        pass


    self.canvas.draw()


if __name__ == '__main__':

    app = QApplication(sys.argv)

    fenetre = FenetrePrincipale()

    fenetre.show()

    sys.exit(app.exec())
```

Votre code est maintenant complet, sécurisé et prêt à l'emploi.

Voulez-vous que je vous montre comment compiler ce script Python en un véritable fichier exécutable (.exe), afin de pouvoir le lancer d'un double-clic sans avoir besoin d'ouvrir le terminal ou VSCode ?

Vous avez dit

Pouvez vous ajouter deux boutons nommés "Export vers JPEG" et "Export vers PNG" ?

Gemini a dit

C'est une excellente idée ! Permettre à l'utilisateur de sauvegarder le graphique généré est une fonctionnalité incontournable pour ce genre d'outil.

Pour faire cela de manière professionnelle, nous allons ajouter deux choses :

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

1. **Des boutons (**QPushButton**)** : Pour déclencher l'action.
2. **Une boîte de dialogue de sauvegarde (**QFileDialog**)** : Pour que l'utilisateur puisse choisir *où* enregistrer l'image sur son ordinateur et sous quel nom.
3. **La fonction** savefig() **de Matplotlib** : C'est elle qui va "prendre la photo" du graphique en haute qualité.

Voici le code complet et mis à jour de votre fichier matplotlib_gemini3_1_pro4.py. Le fichier .json ne change pas du tout.

Le code Python mis à jour

Python

```
import sys

import json

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

# NOUVEAU : Importation de QPushButton et QFileDialog

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QComboBox, QLabel, QHBoxLayout, QMessageBox, QPushButton, QFileDialog


class FenetrePrincipale(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Base de données Animale depuis JSON")

        self.setGeometry(100, 100, 900, 650)
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
# --- 1. CHARGEMENT ET VALIDATION DES DONNÉES ---

self.base_animaux = self.charger_donnees_json("matplotlib_gemini3_1_pro4.json")


if not self.base_animaux:

    return


# --- 2. INTERFACE GRAPHIQUE ---

widget_central = QWidget()

self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


# Menu déroulant et Boutons d'exportation

layout_menu = QHBoxLayout()


label_menu = QLabel("Choisissez une espèce :")

self.menu_deroulant = QComboBox()

self.menu_deroulant.addItems(self.base_animaux.keys())

self.menu_deroulant.currentTextChanged.connect(self.changer_animal)


# NOUVEAU : Création des boutons d'exportation
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
self.btn_export_jpeg = QPushButton("Export vers JPEG")

self.btn_export_png = QPushButton("Export vers PNG")


# Connexion des boutons à notre nouvelle fonction (avec lambda pour passer le format)

self.btn_export_jpeg.clicked.connect(lambda: self.exporter_image("jpeg"))

self.btn_export_png.clicked.connect(lambda: self.exporter_image("png"))


# Ajout au layout du haut

layout_menu.addWidget(label_menu)

layout_menu.addWidget(self.menu_deroulant)

layout_menu.addStretch() # Pousse le menu à gauche et les boutons à droite

layout_menu.addWidget(self.btn_export_jpeg)

layout_menu.addWidget(self.btn_export_png)


layout_principal.addLayout(layout_menu)


# Zone graphique Matplotlib

fig, self.ax = plt.subplots(layout='constrained')

self.canvas = FigureCanvas(fig)

layout_principal.addWidget(self.canvas)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    # Affichage initial
    self.changer_animal(self.menu_deroulant.currentText())

# --- 3. MÉTHODES DE LECTURE ET VALIDATION DU JSON ---
def charger_donnees_json(self, chemin_fichier):
    try:
        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:
            donnees_brutes = json.load(fichier)

            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)

            if liste_erreurs:
                message = "Certaines données du fichier JSON ont été ignorées car elles sont incomplètes :\n\n"
                message += "\n".join(liste_erreurs)
                QMessageBox.warning(self, "Avertissement de Structure", message)

            return donnees_valides

    except FileNotFoundError:
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable. Assurez-vous qu'il est dans le même dossier que le script.")
        return {}
    except json.JSONDecodeError:
        QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON (ex: virgule manquante).")
        return {}

def valider_structure(self, donnees):
    cles_requises = ["categories", "donnees", "unites"]
    donnees_valides = {}
    erreurs = []

    for animal, contenu in donnees.items():
        if not isinstance(contenu, dict):
            erreurs.append(f"- L'entrée '{animal}' n'est pas formatée correctement.")
            continue

        cles_manquantes = [cle for cle in cles_requises if cle not in contenu]

        if cles_manquantes:
            erreurs.append(f"- '{animal}' : il manque les clés ({', '.join(cles_manquantes)})")
```

```
            pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        else:
            donnees_valides[animal] = contenu

    return donnees_valides, erreurs

# --- 4. MÉTHODES DE MISE À JOUR ET DESSIN ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
        titre = f"Comparaison des {nom_espece}"
        self.dessiner_graphique(titre, infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()

    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0

    for attribute, measurement in donnees.items():
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    offset = width * multiplier

    rects = self.ax.bar(x + offset, measurement, width, label=attribute)


    unite = unites.get(attribute, "")

    etiquettes = [f"{val} {unite}" for val in measurement]


    self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)

    multiplier += 1


self.ax.set_title(titre)


if len(donnees) > 0:

    self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)


self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))


try:

    max_val = max([max(v) for v in donnees.values() if v])

    self.ax.set_ylim(0, max_val + (max_val * 0.25))

except ValueError:
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        pass

    self.canvas.draw()

# --- 5. NOUVELLE MÉTHODE : EXPORTATION DE L'IMAGE ---
def exporter_image(self, format_image):
    # On récupère le nom de l'animal actuellement affiché pour proposer un nom de fichier par défaut
    animal_actuel = self.menu_deroulant.currentText()
    nom_fichier_defaut = f"Graphique_{animal_actuel}.{format_image}"

    # Configuration des filtres selon le format choisi
    if format_image == "png":
        filtre = "Images PNG (*.png)"
    else:
        filtre = "Images JPEG (*.jpg *.jpeg)"

    # Ouverture de la fenêtre de sauvegarde PyQt6
    chemin_sauvegarde, _ = QFileDialog.getSaveFileName(
        self,
        f"Sauvegarder en {format_image.upper()}",
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        nom_fichier_defaut,
        filtre
    )

    # Si l'utilisateur n'a pas cliqué sur "Annuler"
    if chemin_sauvegarde:
        try:
            # Sauvegarde avec Matplotlib.
            # dpi=300 donne une image haute définition. bbox_inches='tight' évite que la légende soit coupée.
            self.ax.figure.savefig(chemin_sauvegarde, format=format_image, dpi=300, bbox_inches='tight')
            QMessageBox.information(self, "Succès", f"Graphique exporté avec succès sous :\n{chemin_sauvegarde}")
        except Exception as e:
            QMessageBox.critical(self, "Erreur", f"Une erreur est survenue lors de l'exportation :\n{str(e)}")

if __name__ == '__main__':
    app = QApplication(sys.argv)
    fenetre = FenetrePrincipale()
    fenetre.show()
    sys.exit(app.exec())
```

Qu'avons-nous ajouté ?

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

- QPushButton **et** QFileDialog **:** Les boutons ont été placés en haut à droite (grâce à addStretch() qui agit comme un ressort pour pousser le menu à gauche et les boutons à droite).
- **La fonction générique** exporter_image **:** Plutôt que d'écrire deux fois le même code, j'ai créé une seule fonction qui accepte le format en paramètre ("jpeg" ou "png"). C'est pourquoi nous utilisons lambda: self.exporter_image(...) lors du clic sur le bouton.
- **Nom par défaut intelligent :** Si vous regardez les "Renards", la fenêtre d'enregistrement vous proposera automatiquement le nom Graphique_Renards.png !
- dpi=300 **et** bbox_inches='tight' **:** Ces deux petits paramètres dans savefig sont le secret d'un export professionnel. dpi=300 garantit une image nette pour l'impression, et tight s'assure que la légende à droite ne sera pas tronquée dans l'image finale.

Maintenant que votre outil est fonctionnel et permet l'export, voulez-vous que je vous explique comment transformer tout cela en un véritable fichier exécutable (.exe) ?

Vous avez dit

Pouvez vous ajouter la DATE ET l'HEURE dans les fichiers JPEG et PNG ?

Gemini a dit

C'est une excellente idée ! Pour faire cela de manière vraiment professionnelle, nous pouvons ajouter la date et l'heure à **deux endroits différents** :

1. **Dans le nom du fichier** (ex: Graphique_Renards_2026-04-05_14-30-00.png) pour éviter d'écraser vos anciens exports.
2. **Directement imprimé sur l'image** (comme un petit filigrane gris en bas à droite) pour savoir exactement quand le diagramme a été généré lorsqu'on le regarde.

Pour réaliser cela, nous allons importer le module standard Python nommé datetime.

Voici le code complet mis à jour. (Le fichier .json ne change toujours pas).

Le code Python complet (matplotlib_gemini3_1_pro4.py)

Python

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
import sys

import json

from datetime import datetime # NOUVEAU : Importation du module pour la date et l'heure

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QComboBox, QLabel, QHBoxLayout, QMessageBox, QPushButton, QFileDialog


class FenetrePrincipale(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Base de données Animale depuis JSON")

        self.setGeometry(100, 100, 900, 650)


        # --- 1. CHARGEMENT ET VALIDATION DES DONNÉES ---

        self.base_animaux = self.charger_donnees_json("matplotlib_gemini3_1_pro4.json")


        if not self.base_animaux:

            return
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
# --- 2. INTERFACE GRAPHIQUE ---

widget_central = QWidget()

self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


# Menu déroulant et Boutons d'exportation

layout_menu = QHBoxLayout()


label_menu = QLabel("Choisissez une espèce :")

self.menu_deroulant = QComboBox()

self.menu_deroulant.addItems(self.base_animaux.keys())

self.menu_deroulant.currentTextChanged.connect(self.changer_animal)


self.btn_export_jpeg = QPushButton("Export vers JPEG")

self.btn_export_png = QPushButton("Export vers PNG")


self.btn_export_jpeg.clicked.connect(lambda: self.exporter_image("jpeg"))

self.btn_export_png.clicked.connect(lambda: self.exporter_image("png"))


layout_menu.addWidget(label_menu)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    layout_menu.addWidget(self.menu_deroulant)

    layout_menu.addStretch()

    layout_menu.addWidget(self.btn_export_jpeg)

    layout_menu.addWidget(self.btn_export_png)


    layout_principal.addLayout(layout_menu)


    # Zone graphique Matplotlib

    fig, self.ax = plt.subplots(layout='constrained')

    self.canvas = FigureCanvas(fig)

    layout_principal.addWidget(self.canvas)


    # Affichage initial

    self.changer_animal(self.menu_deroulant.currentText())


# --- 3. MÉTHODES DE LECTURE ET VALIDATION DU JSON ---

def charger_donnees_json(self, chemin_fichier):

    try:

        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:

            donnees_brutes = json.load(fichier)
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)

            if liste_erreurs:
                message = "Certaines données du fichier JSON ont été ignorées car elles sont incomplètes :\n\n"
                message += "\n".join(liste_erreurs)
                QMessageBox.warning(self, "Avertissement de Structure", message)

            return donnees_valides

        except FileNotFoundError:
            QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable. Assurez-vous qu'il est dans le même dossier que le script.")
            return {}
        except json.JSONDecodeError:
            QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON (ex: virgule manquante).")
            return {}

    def valider_structure(self, donnees):
        cles_requises = ["categories", "donnees", "unites"]
        donnees_valides = {}
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    erreurs = []

    for animal, contenu in donnees.items():
        if not isinstance(contenu, dict):
            erreurs.append(f"- L'entrée '{animal}' n'est pas formatée correctement.")
            continue

        cles_manquantes = [cle for cle in cles_requises if cle not in contenu]

        if cles_manquantes:
            erreurs.append(f"- '{animal}' : il manque les clés ({', '.join(cles_manquantes)})")
        else:
            donnees_valides[animal] = contenu

    return donnees_valides, erreurs

# --- 4. MÉTHODES DE MISE À JOUR ET DESSIN ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        titre = f"Comparaison des {nom_espece}"

        self.dessiner_graphique(titre, infos["categories"], infos["donnees"], infos["unites"])


def dessiner_graphique(self, titre, categories, donnees, unites):

    self.ax.clear()


    x = np.arange(len(categories))

    width = 0.25

    multiplier = 0


    for attribute, measurement in donnees.items():

        offset = width * multiplier

        rects = self.ax.bar(x + offset, measurement, width, label=attribute)


        unite = unites.get(attribute, "")

        etiquettes = [f"{val} {unite}" for val in measurement]


        self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)

        multiplier += 1
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    self.ax.set_title(titre)

    if len(donnees) > 0:
        self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)

    self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))

    try:
        max_val = max([max(v) for v in donnees.values() if v])
        self.ax.set_ylim(0, max_val + (max_val * 0.25))
    except ValueError:
        pass

    self.canvas.draw()

# --- 5. MÉTHODE D'EXPORTATION MISE À JOUR ---
def exporter_image(self, format_image):
    animal_actuel = self.menu_deroulant.currentText()

    # NOUVEAU : Récupération de l'heure exacte
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
maintenant = datetime.now()

# Format pour le nom du fichier (sans espaces ni deux-points car interdits dans les noms de fichiers Windows)

date_heure_fichier = maintenant.strftime("%Y-%m-%d_%Hh%Mm%Ss")

# Format lisible pour le texte sur l'image

date_heure_texte = maintenant.strftime("%d/%m/%Y à %H:%M:%S")


# Le nom du fichier inclut maintenant la date et l'heure

nom_fichier_defaut = f"Graphique_{animal_actuel}_{date_heure_fichier}.{format_image}"


if format_image == "png":

    filtre = "Images PNG (*.png)"

else:

    filtre = "Images JPEG (*.jpg *.jpeg)"


chemin_sauvegarde, _ = QFileDialog.getSaveFileName(

    self,

    f"Sauvegarder en {format_image.upper()}",

    nom_fichier_defaut,

    filtre

)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        if chemin_sauvegarde:
            try:
                # NOUVEAU : On écrit la date en petit gris en bas à droite de l'image
                texte_filigrane = self.ax.figure.text(0.99, 0.01, f"Généré le {date_heure_texte}",
                                                      ha='right', va='bottom', fontsize=8, color='gray')

                # On sauvegarde l'image avec ce texte
                self.ax.figure.savefig(chemin_sauvegarde, format=format_image, dpi=300, bbox_inches='tight')

                # NOUVEAU : On efface le texte de l'interface juste après l'export,
                # pour qu'il ne reste pas affiché dans le logiciel lui-même.
                texte_filigrane.remove()
                self.canvas.draw()

                QMessageBox.information(self, "Succès", f"Graphique exporté avec succès sous :\n{chemin_sauvegarde}")
            except Exception as e:
                QMessageBox.critical(self, "Erreur", f"Une erreur est survenue lors de l'exportation :\n{str(e)}")

if __name__ == '__main__':
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
app = QApplication(sys.argv)

fenetre = FenetrePrincipale()

fenetre.show()

sys.exit(app.exec())
```

Explication des nouveautés :

- datetime.now().strftime(...) : La fonction strftime (String Format Time) permet de transformer une date en texte selon un format précis.
    - Pour le **nom du fichier**, j'ai utilisé %Y-%m-%d_%Hh%Mm%Ss (ex: 2026-04-05_14h30m15s). Windows interdit d'utiliser des deux-points : dans le nom d'un fichier, cette mise en forme contourne donc le problème élégamment.
    - Pour le **texte sur l'image**, j'ai utilisé le format classique %d/%m/%Y à %H:%M:%S (ex: 05/04/2026 à 14:30:15).
- self.ax.figure.text(...) : Cette ligne ajoute la date directement sur le diagramme, en bas à droite (coordonnées 0.99, 0.01).
- texte_filigrane.remove() : C'est une petite astuce technique ! On ajoute la date sur le graphique juste avant de prendre la "photo" (savefig), puis on l'efface tout de suite après avec remove(). De cette façon, le filigrane n'apparaît que sur le fichier exporté, sans polluer la fenêtre de votre logiciel.

Souhaitez-vous que l'on passe à l'étape finale qui consisterait à compiler tout cela en fichier exécutable ?

Vous avez dit

Maintenant pouvez vous ajouter une DATE et HEURE à l'intérieur du Fichier JSON puis afficher la DATE ET HEURE contenues dans le Fichier JSON sur le logiciel PyQt6 afin de pouvoir dater avec date et heure de collecte du Fichier JSON ? svp

Gemini a dit

C'est une excellente pratique ! En entreprise, on appelle cela la **traçabilité des données**. Il est crucial de savoir de quand datent les informations que l'on est en train d'analyser.

Pour réaliser cela sans "casser" notre programme (car le programme s'attend à ce que chaque élément du JSON soit un animal), nous allons ajouter une clé spéciale nommée "_Metadata" tout au début du fichier JSON. Le tiret du bas (_) est une convention classique en programmation pour indiquer qu'il s'agit d'une donnée de configuration et non d'une donnée à afficher dans le graphique.

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Voici les deux fichiers mis à jour :

Fichier 1 : matplotlib_gemini3_1_pro4.json

Nous avons ajouté le bloc _Metadata tout en haut avec la date et l'heure de collecte.

JSON


```
{
  "_Metadata": {
    "date_collecte": "05/04/2026 à 11:30:00"
  },
  "Pingouins": {
    "categories": ["Adelie", "Chinstrap", "Gentoo"],
    "donnees": {
      "Profondeur du bec": [18.35, 18.43, 14.98],
      "Longueur du bec": [38.79, 48.83, 47.50],
      "Longueur des nageoires": [189.95, 195.82, 217.19]
    },
    "unites": {
      "Profondeur du bec": "mm",
      "Longueur du bec": "mm",
      "Longueur des nageoires": "mm"
    }
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py


```
},
"Renards": {
  "categories": ["Polaire", "Roux", "Fennec"],
  "donnees": {
    "Taille moyenne": [55.0, 65.5, 38.0],
    "Poids moyen": [4.5, 6.0, 1.2],
    "Vitesse max": [45.0, 50.0, 32.0]
  },
  "unites": {
    "Taille moyenne": "cm",
    "Poids moyen": "kg",
    "Vitesse max": "km/h"
  }
},
"Félins": {
  "categories": ["Lion", "Tigre", "Guépard"],
  "donnees": {
    "Poids": [190.0, 220.0, 50.0],
    "Vitesse max": [80.0, 65.0, 120.0]
  },
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

```
      "unites": {
        "Poids": "kg",
        "Vitesse max": "km/h"
      }
    }
}
```

Fichier 2 : matplotlib_gemini3_1_pro4.py

J'ai ajouté un bloc de texte stylisé (QLabel) en haut de l'interface graphique pour afficher cette date, et j'ai modifié la fonction de validation pour qu'elle ignore intelligemment le bloc _Metadata lorsqu'elle vérifie les animaux.

Python

```
import sys
import json
from datetime import datetime
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas
from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QComboBox, QLabel, QHBoxLayout, QMessageBox, QPushButton, QFileDialog


class FenetrePrincipale(QMainWindow):
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
def __init__(self):
    super().__init__()
    self.setWindowTitle("Base de données Animale depuis JSON")
    self.setGeometry(100, 100, 900, 650)

    # --- 1. CHARGEMENT ET VALIDATION DES DONNÉES ---
    donnees_completes = self.charger_donnees_json("matplotlib_gemini3_1_pro4.json")

    if not donnees_completes:
        return

    # NOUVEAU : On extrait la date de collecte (et on la retire du dictionnaire des animaux avec .pop)
    metadata = donnees_completes.pop("_Metadata", {})
    date_collecte = metadata.get("date_collecte", "Date non spécifiée")

    # Tout ce qui reste dans le dictionnaire ce sont les animaux
    self.base_animaux = donnees_completes

    # --- 2. INTERFACE GRAPHIQUE ---
    widget_central = QWidget()
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


# NOUVEAU : Ajout d'une étiquette pour afficher la date de collecte du JSON en haut de la fenêtre

label_date_collecte = QLabel(f"📅 Fichier de données collecté le : {date_collecte}")

label_date_collecte.setStyleSheet("color: #666666; font-style: italic; font-size: 10pt; margin-bottom: 5px;")

layout_principal.addWidget(label_date_collecte)


# Menu déroulant et Boutons d'exportation

layout_menu = QHBoxLayout()


label_menu = QLabel("Choisissez une espèce :")

self.menu_deroulant = QComboBox()

self.menu_deroulant.addItems(self.base_animaux.keys())

self.menu_deroulant.currentTextChanged.connect(self.changer_animal)


self.btn_export_jpeg = QPushButton("Export vers JPEG")

self.btn_export_png = QPushButton("Export vers PNG")


self.btn_export_jpeg.clicked.connect(lambda: self.exporter_image("jpeg"))
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    self.btn_export_png.clicked.connect(lambda: self.exporter_image("png"))

    layout_menu.addWidget(label_menu)
    layout_menu.addWidget(self.menu_deroulant)
    layout_menu.addStretch()
    layout_menu.addWidget(self.btn_export_jpeg)
    layout_menu.addWidget(self.btn_export_png)

    layout_principal.addLayout(layout_menu)

    # Zone graphique Matplotlib
    fig, self.ax = plt.subplots(layout='constrained')
    self.canvas = FigureCanvas(fig)
    layout_principal.addWidget(self.canvas)

    # Affichage initial
    self.changer_animal(self.menu_deroulant.currentText())

# --- 3. MÉTHODES DE LECTURE ET VALIDATION DU JSON ---
def charger_donnees_json(self, chemin_fichier):
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
try:
    with open(chemin_fichier, 'r', encoding='utf-8') as fichier:
        donnees_brutes = json.load(fichier)

        donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)

        if liste_erreurs:
            message = "Certaines données du fichier JSON ont été ignorées car elles sont incomplètes :\n\n"
            message += "\n".join(liste_erreurs)
            QMessageBox.warning(self, "Avertissement de Structure", message)

        return donnees_valides

except FileNotFoundError:
    QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable. Assurez-vous qu'il est dans le même dossier que le script.")
    return {}
except json.JSONDecodeError:
    QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON (ex: virgule manquante).")
    return {}
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
def valider_structure(self, donnees):
    cles_requises = ["categories", "donnees", "unites"]
    donnees_valides = {}
    erreurs = []

    for cle, contenu in donnees.items():
        # NOUVEAU : On isole la clé _Metadata pour qu'elle ne soit pas considérée comme un animal invalide
        if cle == "_Metadata":
            donnees_valides[cle] = contenu
            continue

        if not isinstance(contenu, dict):
            erreurs.append(f"- L'entrée '{cle}' n'est pas formatée correctement.")
            continue

        cles_manquantes = [c for c in cles_requises if c not in contenu]

        if cles_manquantes:
            erreurs.append(f"- '{cle}' : il manque les clés ({', '.join(cles_manquantes)})")
        else:
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            donnees_valides[cle] = contenu

    return donnees_valides, erreurs

# --- 4. MÉTHODES DE MISE À JOUR ET DESSIN ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
        titre = f"Comparaison des {nom_espece}"
        self.dessiner_graphique(titre, infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()

    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0

    for attribute, measurement in donnees.items():
        offset = width * multiplier
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    rects = self.ax.bar(x + offset, measurement, width, label=attribute)

    unite = unites.get(attribute, "")
    etiquettes = [f"{val} {unite}" for val in measurement]

    self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)
    multiplier += 1

self.ax.set_title(titre)

if len(donnees) > 0:
    self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)

self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))

try:
    max_val = max([max(v) for v in donnees.values() if v])
    self.ax.set_ylim(0, max_val + (max_val * 0.25))
except ValueError:
    pass
```

```
    self.canvas.draw()

# --- 5. MÉTHODE D'EXPORTATION ---
def exporter_image(self, format_image):
    animal_actuel = self.menu_deroulant.currentText()
    maintenant = datetime.now()

    date_heure_fichier = maintenant.strftime("%Y-%m-%d_%Hh%Mm%Ss")
    date_heure_texte = maintenant.strftime("%d/%m/%Y à %H:%M:%S")

    nom_fichier_defaut = f"Graphique_{animal_actuel}_{date_heure_fichier}.{format_image}"

    if format_image == "png":
        filtre = "Images PNG (*.png)"
    else:
        filtre = "Images JPEG (*.jpg *.jpeg)"

    chemin_sauvegarde, _ = QFileDialog.getSaveFileName(
        self,
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        f"Sauvegarder en {format_image.upper()}",
        nom_fichier_defaut,
        filtre
    )

    if chemin_sauvegarde:
        try:
            texte_filigrane = self.ax.figure.text(0.99, 0.01, f"Généré le {date_heure_texte}",
                                                  ha='right', va='bottom', fontsize=8, color='gray')

            self.ax.figure.savefig(chemin_sauvegarde, format=format_image, dpi=300, bbox_inches='tight')

            texte_filigrane.remove()
            self.canvas.draw()

            QMessageBox.information(self, "Succès", f"Graphique exporté avec succès sous :\n{chemin_sauvegarde}")
        except Exception as e:
            QMessageBox.critical(self, "Erreur", f"Une erreur est survenue lors de l'exportation :\n{str(e)}")

if __name__ == '__main__':
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

app = QApplication(sys.argv)

fenetre = FenetrePrincipale()

fenetre.show()

sys.exit(app.exec())

Comment ça marche :

- donnees_completes.pop("_Metadata", {}) : La méthode .pop() est magique ici. Elle va chercher la clé _Metadata dans les données, l'enregistre dans une variable séparée, **et la supprime** du dictionnaire principal ! Ainsi, quand PyQt6 va remplir le menu déroulant avec les clés restantes, il ne verra plus _Metadata et n'affichera que les vrais animaux.
- label_date_collecte.setStyleSheet(...) : J'ai ajouté un peu de style (CSS) pour que la date soit grise et en italique, afin qu'elle fasse vraiment "information secondaire" en haut de la fenêtre.

Vous avez dit

Mon fichier JSON s'apelle matplotlib_gemini3_1_pro5.json

Gemini a dit

C'est noté ! C'est une modification très simple à faire.

Il suffit de changer le nom du fichier à l'endroit précis où le programme tente de le charger, c'est-à-dire dans la méthode d'initialisation de votre interface graphique.

Voici le code Python complet avec la mise à jour appliquée (j'ai modifié la ligne self.charger_donnees_json("matplotlib_gemini3_1_pro5.json")). Vous pouvez remplacer tout le contenu de votre fichier .py par celui-ci :

Python

import sys

import json

from datetime import datetime

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QComboBox, QLabel, QHBoxLayout, QMessageBox, QPushButton, QFileDialog


class FenetrePrincipale(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Base de données Animale depuis JSON")

        self.setGeometry(100, 100, 900, 650)


        # --- 1. CHARGEMENT ET VALIDATION DES DONNÉES ---

        # MISE À JOUR : Le nom du fichier correspond maintenant à votre version !

        donnees_completes = self.charger_donnees_json("matplotlib_gemini3_1_pro5.json")


        if not donnees_completes:

            return


        metadata = donnees_completes.pop("_Metadata", {})

        date_collecte = metadata.get("date_collecte", "Date non spécifiée")
```

```
self.base_animaux = donnees_completes

# --- 2. INTERFACE GRAPHIQUE ---
widget_central = QWidget()
self.setCentralWidget(widget_central)
layout_principal = QVBoxLayout(widget_central)

label_date_collecte = QLabel(f"📅 Fichier de données collecté le : {date_collecte}")
label_date_collecte.setStyleSheet("color: #666666; font-style: italic; font-size: 10pt; margin-bottom: 5px;")
layout_principal.addWidget(label_date_collecte)

layout_menu = QHBoxLayout()

label_menu = QLabel("Choisissez une espèce :")
self.menu_deroulant = QComboBox()
self.menu_deroulant.addItems(self.base_animaux.keys())
self.menu_deroulant.currentTextChanged.connect(self.changer_animal)

self.btn_export_jpeg = QPushButton("Export vers JPEG")
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    self.btn_export_png = QPushButton("Export vers PNG")

    self.btn_export_jpeg.clicked.connect(lambda: self.exporter_image("jpeg"))
    self.btn_export_png.clicked.connect(lambda: self.exporter_image("png"))

    layout_menu.addWidget(label_menu)
    layout_menu.addWidget(self.menu_deroulant)
    layout_menu.addStretch()
    layout_menu.addWidget(self.btn_export_jpeg)
    layout_menu.addWidget(self.btn_export_png)

    layout_principal.addLayout(layout_menu)

    fig, self.ax = plt.subplots(layout='constrained')
    self.canvas = FigureCanvas(fig)
    layout_principal.addWidget(self.canvas)

    self.changer_animal(self.menu_deroulant.currentText())

# --- 3. MÉTHODES DE LECTURE ET VALIDATION DU JSON ---
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
def charger_donnees_json(self, chemin_fichier):
    try:
        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:
            donnees_brutes = json.load(fichier)

            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)

            if liste_erreurs:
                message = "Certaines données du fichier JSON ont été ignorées car elles sont incomplètes :\n\n"
                message += "\n".join(liste_erreurs)
                QMessageBox.warning(self, "Avertissement de Structure", message)

            return donnees_valides

    except FileNotFoundError:
        QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable. Assurez-vous qu'il est dans le même dossier que le script.")
        return {}
    except json.JSONDecodeError:
        QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON (ex: virgule manquante).")
        return {}
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
def valider_structure(self, donnees):
    cles_requises = ["categories", "donnees", "unites"]
    donnees_valides = {}
    erreurs = []

    for cle, contenu in donnees.items():
        if cle == "_Metadata":
            donnees_valides[cle] = contenu
            continue

        if not isinstance(contenu, dict):
            erreurs.append(f"- L'entrée '{cle}' n'est pas formatée correctement.")
            continue

        cles_manquantes = [c for c in cles_requises if c not in contenu]

        if cles_manquantes:
            erreurs.append(f"- '{cle}' : il manque les clés ({', '.join(cles_manquantes)})")
        else:
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            donnees_valides[cle] = contenu

    return donnees_valides, erreurs

# --- 4. MÉTHODES DE MISE À JOUR ET DESSIN ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
        titre = f"Comparaison des {nom_espece}"
        self.dessiner_graphique(titre, infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()

    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0

    for attribute, measurement in donnees.items():
        offset = width * multiplier
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    rects = self.ax.bar(x + offset, measurement, width, label=attribute)

    unite = unites.get(attribute, "")
    etiquettes = [f"{val} {unite}" for val in measurement]

    self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)
    multiplier += 1

self.ax.set_title(titre)

if len(donnees) > 0:
    self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)

self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))

try:
    max_val = max([max(v) for v in donnees.values() if v])
    self.ax.set_ylim(0, max_val + (max_val * 0.25))
except ValueError:
    pass
```

```
    self.canvas.draw()

# --- 5. MÉTHODE D'EXPORTATION ---
def exporter_image(self, format_image):
    animal_actuel = self.menu_deroulant.currentText()
    maintenant = datetime.now()

    date_heure_fichier = maintenant.strftime("%Y-%m-%d_%Hh%Mm%Ss")
    date_heure_texte = maintenant.strftime("%d/%m/%Y à %H:%M:%S")

    nom_fichier_defaut = f"Graphique_{animal_actuel}_{date_heure_fichier}.{format_image}"

    if format_image == "png":
        filtre = "Images PNG (*.png)"
    else:
        filtre = "Images JPEG (*.jpg *.jpeg)"

    chemin_sauvegarde, _ = QFileDialog.getSaveFileName(
        self,
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        f"Sauvegarder en {format_image.upper()}",
        nom_fichier_defaut,
        filtre
    )

    if chemin_sauvegarde:
        try:
            texte_filigrane = self.ax.figure.text(0.99, 0.01, f"Généré le {date_heure_texte}",
                                                  ha='right', va='bottom', fontsize=8, color='gray')

            self.ax.figure.savefig(chemin_sauvegarde, format=format_image, dpi=300, bbox_inches='tight')

            texte_filigrane.remove()
            self.canvas.draw()

            QMessageBox.information(self, "Succès", f"Graphique exporté avec succès sous :\n{chemin_sauvegarde}")
        except Exception as e:
            QMessageBox.critical(self, "Erreur", f"Une erreur est survenue lors de l'exportation :\n{str(e)}")

if __name__ == '__main__':
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
app = QApplication(sys.argv)

fenetre = FenetrePrincipale()

fenetre.show()

sys.exit(app.exec())
```

Votre code est maintenant parfaitement aligné avec le nom de votre fichier de données.

Avez-vous pu tester le rendu final ? Souhaitez-vous que l'on passe à la compilation de ce script en fichier exécutable (.exe) pour votre portfolio GitHub ?

Vous avez dit

Pouvez vous mettre la phrase : Fichier de données collecté le : à l'intérieur des fichiers JPEG ET PNG ? svp

Gemini a dit

C'est une excellente idée pour assurer la traçabilité complète de vos données, même une fois le graphique exporté en image.

Pour réaliser cela, nous devons effectuer deux petites modifications dans le code Python :

1. **Mémoriser la date du JSON** : Dans le constructeur (__init__), nous allons stocker la date de collecte lue dans le fichier JSON dans une variable d'instance (accessible partout dans la classe).
2. **L'ajouter lors de l'export** : Dans la méthode exporter_image, nous allons ajouter ce texte sur la figure Matplotlib juste avant de sauvegarder le fichier, exactement comme nous l'avons fait pour la date de génération.

Voici le code complet et mis à jour de votre fichier matplotlib_gemini3_1_pro5.py.

Le code Python complet

Python

```
import sys

import json
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
from datetime import datetime

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QComboBox, QLabel, QHBoxLayout, QMessageBox, QPushButton, QFileDialog


class FenetrePrincipale(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Base de données Animale depuis JSON")

        self.setGeometry(100, 100, 900, 650)


        # --- 1. CHARGEMENT ET VALIDATION DES DONNÉES ---

        donnees_completes = self.charger_donnees_json("matplotlib_gemini3_1_pro5.json")


        if not donnees_completes:

            return


        metadata = donnees_completes.pop("_Metadata", {})

        # NOUVEAU : On stocke la date dans self.date_collecte_json pour l'utiliser lors de l'export
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
self.date_collecte_json = metadata.get("date_collecte", "Date non spécifiée")

self.base_animaux = donnees_completes

# --- 2. INTERFACE GRAPHIQUE ---
widget_central = QWidget()
self.setCentralWidget(widget_central)
layout_principal = QVBoxLayout(widget_central)

# Affichage de la date de collecte en haut de la fenêtre
label_date_collecte = QLabel(f"📅 Fichier de données collecté le : {self.date_collecte_json}")
label_date_collecte.setStyleSheet("color: #666666; font-style: italic; font-size: 10pt; margin-bottom: 5px;")
layout_principal.addWidget(label_date_collecte)

layout_menu = QHBoxLayout()

label_menu = QLabel("Choisissez une espèce :")
self.menu_deroulant = QComboBox()
self.menu_deroulant.addItems(self.base_animaux.keys())
self.menu_deroulant.currentTextChanged.connect(self.changer_animal)
```

```
self.btn_export_jpeg = QPushButton("Export vers JPEG")
self.btn_export_png = QPushButton("Export vers PNG")

self.btn_export_jpeg.clicked.connect(lambda: self.exporter_image("jpeg"))
self.btn_export_png.clicked.connect(lambda: self.exporter_image("png"))

layout_menu.addWidget(label_menu)
layout_menu.addWidget(self.menu_deroulant)
layout_menu.addStretch()
layout_menu.addWidget(self.btn_export_jpeg)
layout_menu.addWidget(self.btn_export_png)

layout_principal.addLayout(layout_menu)

fig, self.ax = plt.subplots(layout='constrained')
self.canvas = FigureCanvas(fig)
layout_principal.addWidget(self.canvas)

self.changer_animal(self.menu_deroulant.currentText())
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

# --- 3. MÉTHODES DE LECTURE ET VALIDATION DU JSON ---
def charger_donnees_json(self, chemin_fichier):
    try:
        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:
            donnees_brutes = json.load(fichier)

            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)

            if liste_erreurs:
                message = "Certaines données du fichier JSON ont été ignorées car elles sont incomplètes :\n\n"
                message += "\n".join(liste_erreurs)
                QMessageBox.warning(self, "Avertissement de Structure", message)

            return donnees_valides

    except FileNotFoundError:
        QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable. Assurez-vous qu'il est dans le même dossier que le script.")
        return {}
    except json.JSONDecodeError:
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        QMessageBox.critical(self, "Erreur de syntaxe", f"Le fichier {chemin_fichier} contient des erreurs de syntaxe JSON (ex: virgule manquante).")
        return {}

    def valider_structure(self, donnees):
        cles_requises = ["categories", "donnees", "unites"]
        donnees_valides = {}
        erreurs = []

        for cle, contenu in donnees.items():
            if cle == "_Metadata":
                donnees_valides[cle] = contenu
                continue

            if not isinstance(contenu, dict):
                erreurs.append(f"- L'entrée '{cle}' n'est pas formatée correctement.")
                continue

            cles_manquantes = [c for c in cles_requises if c not in contenu]

            if cles_manquantes:
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            erreurs.append(f"- '{cle}' : il manque les clés ({', '.join(cles_manquantes)})")
        else:
            donnees_valides[cle] = contenu

    return donnees_valides, erreurs

# --- 4. MÉTHODES DE MISE À JOUR ET DESSIN ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
        titre = f"Comparaison des {nom_espece}"
        self.dessiner_graphique(titre, infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()

    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
for attribute, measurement in donnees.items():
    offset = width * multiplier
    rects = self.ax.bar(x + offset, measurement, width, label=attribute)

    unite = unites.get(attribute, "")
    etiquettes = [f"{val} {unite}" for val in measurement]

    self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)
    multiplier += 1

self.ax.set_title(titre)

if len(donnees) > 0:
    self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)

self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))

try:
    max_val = max([max(v) for v in donnees.values() if v])
    self.ax.set_ylim(0, max_val + (max_val * 0.25))
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
    except ValueError:
        pass

    self.canvas.draw()

# --- 5. MÉTHODE D'EXPORTATION MISE À JOUR ---
def exporter_image(self, format_image):
    animal_actuel = self.menu_deroulant.currentText()
    maintenant = datetime.now()

    date_heure_fichier = maintenant.strftime("%Y-%m-%d_%Hh%Mm%Ss")
    date_heure_texte = maintenant.strftime("%d/%m/%Y à %H:%M:%S")

    nom_fichier_defaut = f"Graphique_{animal_actuel}_{date_heure_fichier}.{format_image}"

    if format_image == "png":
        filtre = "Images PNG (*.png)"
    else:
        filtre = "Images JPEG (*.jpg *.jpeg)"
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
chemin_sauvegarde, _ = QFileDialog.getSaveFileName(
    self,
    f"Sauvegarder en {format_image.upper()}",
    nom_fichier_defaut,
    filtre
)

if chemin_sauvegarde:
    try:
        # NOUVEAU (1/2) : On ajoute la phrase de collecte en haut à gauche de l'image (en gris clair)
        phrase_collecte = f"Fichier de données collecté le : {self.date_collecte_json}"
        texte_collecte = self.ax.figure.text(0.01, 0.99, phrase_collecte,
                                             ha='left', va='top', fontsize=8, color='#888888', fontstyle='italic')

        # NOUVEAU (2/2) : On conserve aussi la date de génération en bas à droite
        texte_generation = self.ax.figure.text(0.99, 0.01, f"Image générée le {date_heure_texte}",
                                               ha='right', va='bottom', fontsize=8, color='#888888')

        # Sauvegarde de l'image avec les deux textes
        self.ax.figure.savefig(chemin_sauvegarde, format=format_image, dpi=300, bbox_inches='tight')
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            # On nettoie la figure pour l'interface logicielle
            texte_collecte.remove()
            texte_generation.remove()
            self.canvas.draw()

            QMessageBox.information(self, "Succès", f"Graphique exporté avec succès sous :\n{chemin_sauvegarde}")
        except Exception as e:
            QMessageBox.critical(self, "Erreur", f"Une erreur est survenue lors de l'exportation :\n{str(e)}")

if __name__ == '__main__':
    app = QApplication(sys.argv)
    fenetre = FenetrePrincipale()
    fenetre.show()
    sys.exit(app.exec())
```

Ce qui va se passer maintenant :

Lorsque vous exporterez une image (JPEG ou PNG) :

1. **En haut à gauche**, en petit, gris et italique, il y aura écrit : Fichier de données collecté le : [date du JSON].
2. **En bas à droite**, il y aura toujours écrit : Image générée le [date et heure actuelle].

Vous avez maintenant un graphique doublement daté, offrant une traçabilité parfaite !

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Vous avez dit

Est t il possible d'envoyer les fichiers JPEG et PNG et JSON dans un EMAIL avec GMAIL en supposant que le compte gmailpossède déjà un MOT DE PASSE D'APPLICATION?

Gemini a dit

Oui, c'est tout à fait possible et c'est même très courant ! Puisque vous avez déjà configuré un mot de passe d'application Gmail, le plus dur est fait.

Python possède des bibliothèques intégrées (smtplib et email) qui permettent de se connecter aux serveurs de Google et de construire un email complexe contenant du texte et des pièces jointes (vos fichiers JPEG, PNG et JSON).

Voici un script Python complet et prêt à l'emploi. Vous pouvez le tester dans un fichier séparé (par exemple test_email.py) avant de l'intégrer à votre logiciel principal.

Le code d'envoi d'email avec pièces jointes

Python

```
import smtplib

from email.mime.multipart import MIMEMultipart

from email.mime.text import MIMEText

from email.mime.base import MIMEBase

from email import encoders

import os


def envoyer_email_avec_pieces_jointes(expediteur, mot_de_passe, destinataire, sujet, corps_message, chemins_fichiers):

    # 1. Création du message

    msg = MIMEMultipart()
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
msg['From'] = expediteur

msg['To'] = destinataire

msg['Subject'] = sujet


# 2. Ajout du texte de l'email

msg.attach(MIMEText(corps_message, 'plain'))


# 3. Traitement et ajout des pièces jointes

for chemin in chemins_fichiers:

    if os.path.exists(chemin):

        try:

            # On ouvre le fichier en mode binaire (rb)

            with open(chemin, "rb") as fichier:

                part = MIMEBase("application", "octet-stream")

                part.set_payload(fichier.read())


            # Encodage en base64 pour que ça passe par email

            encoders.encode_base64(part)


            # Ajout de l'en-tête pour que le fichier ait le bon nom à la réception
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            nom_fichier = os.path.basename(chemin)

            part.add_header("Content-Disposition", f"attachment; filename={nom_fichier}")


            msg.attach(part)

            print(f"✅ Pièce jointe ajoutée : {nom_fichier}")

        except Exception as e:

            print(f"❌ Erreur avec le fichier {chemin} : {e}")

    else:

        print(f"⚠️ Le fichier {chemin} est introuvable.")


# 4. Connexion au serveur Gmail et envoi

try:

    print("Connexion au serveur Gmail...")

    serveur = smtplib.SMTP('smtp.gmail.com', 587)

    serveur.starttls() # Sécurisation de la connexion

    serveur.login(expediteur, mot_de_passe)


    texte_final = msg.as_string()

    serveur.sendmail(expediteur, destinataire, texte_final)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        serveur.quit()

        print("🚀 Email envoyé avec succès !")


    except Exception as e:

        print(f"❌ Erreur lors de l'envoi de l'email : {e}")


# ==========================================

# EXEMPLE D'UTILISATION

# ==========================================

if __name__ == '__main__':

    # Remplissez vos informations ici

    MON_EMAIL = "votre.email@gmail.com"

    MON_MOT_DE_PASSE_APP = "xxxx xxxx xxxx xxxx" # Les 16 lettres générées par Google

    EMAIL_DESTINATAIRE = "destinataire@exemple.com"


    SUJET = "Export des données de l'application"

    CORPS = "Bonjour,\n\nVeuillez trouver ci-joint les graphiques générés (PNG/JPEG) ainsi que le fichier de données source (JSON).\n\nCordialement,"


    # La liste des fichiers que vous voulez envoyer

    LISTE_FICHIERS = [
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
    "matplotlib_gemini3_1_pro5.json",
    "Graphique_Renards_2026-04-05_12h00m00s.png" # Remplacez par le nom d'un graphique que vous avez exporté
]

envoyer_email_avec_pieces_jointes(MON_EMAIL, MON_MOT_DE_PASSE_APP, EMAIL_DESTINATAIRE, SUJET, CORPS, LISTE_FICHIERS)
```

🔒 **Un conseil de sécurité crucial**

Pour garder votre code parfaitement sécurisé lors de vos futurs *push* ou partages sur vos dépôts Git, n'écrivez jamais ce mot de passe d'application en clair directement dans votre fichier .py.

La meilleure pratique consiste à utiliser un fichier d'environnement externe (comme un fichier .env) que vous excluez de vos commits, ou à utiliser une petite fenêtre de dialogue PyQt6 qui vous demande de taper le mot de passe au moment de l'envoi.

Voulez-vous que je vous montre comment intégrer directement ce mécanisme d'email dans notre logiciel, avec un bouton "Envoyer par Email" qui ouvre une petite fenêtre pour saisir l'adresse du destinataire ?

Vous avez dit

Oui

Gemini a dit

C'est parti ! Nous allons créer une petite fenêtre de dialogue qui s'ouvrira lorsque vous cliquerez sur le bouton **"Envoyer par Email"**.

Cette approche est la plus sûre :

1. Elle vous demande votre mot de passe d'application uniquement au moment de l'envoi, en le masquant par des points noirs (***), pour des raisons de sécurité.
2. Elle vous ouvre ensuite une fenêtre pour sélectionner les fichiers exacts que vous voulez joindre (votre JSON, et les images que vous avez exportées).

Voici le code complet de votre fichier matplotlib_gemini3_1_pro5.py mis à jour avec cette nouvelle fonctionnalité.

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Le code Python complet

Python

```
import sys

import json

import os

import smtplib

from email.mime.multipart import MIMEMultipart

from email.mime.text import MIMEText

from email.mime.base import MIMEBase

from email import encoders

from datetime import datetime


import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

# NOUVEAU : Importation de QDialog, QFormLayout, QLineEdit, QDialogButtonBox

from PyQt6.QtWidgets import (QApplication, QMainWindow, QVBoxLayout, QWidget,

                             QComboBox, QLabel, QHBoxLayout, QMessageBox,

                             QPushButton, QFileDialog, QDialog, QFormLayout,

                             QLineEdit, QDialogButtonBox)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
# --- NOUVELLE CLASSE : Fenêtre de dialogue pour l'email ---
class DialogueEmail(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Configuration de l'Email")
        self.setMinimumWidth(400)

        layout = QFormLayout(self)

        self.champ_expediteur = QLineEdit(self)
        self.champ_expediteur.setPlaceholderText("votre.email@gmail.com")

        self.champ_mdp = QLineEdit(self)
        # On masque le mot de passe pour la sécurité
        self.champ_mdp.setEchoMode(QLineEdit.EchoMode.Password)
        self.champ_mdp.setPlaceholderText("Mot de passe d'application (16 lettres)")

        self.champ_destinataire = QLineEdit(self)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        self.champ_destinataire.setPlaceholderText("destinataire@exemple.com")

        layout.addRow("Votre email Gmail :", self.champ_expediteur)
        layout.addRow("Mot de passe d'app :", self.champ_mdp)
        layout.addRow("Email destinataire :", self.champ_destinataire)

        # Boutons OK et Annuler
        boutons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok | QDialogButtonBox.StandardButton.Cancel)
        boutons.accepted.connect(self.accept)
        boutons.rejected.connect(self.reject)
        layout.addWidget(boutons)

    def obtenir_donnees(self):
        return self.champ_expediteur.text(), self.champ_mdp.text(), self.champ_destinataire.text()


# --- CLASSE PRINCIPALE ---
class FenetrePrincipale(QMainWindow):
    def __init__(self):
        super().__init__()
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
self.setWindowTitle("Base de données Animale depuis JSON")

self.setGeometry(100, 100, 1000, 650) # J'ai élargi un peu la fenêtre pour les boutons


# 1. CHARGEMENT ET VALIDATION DES DONNÉES

donnees_completes = self.charger_donnees_json("matplotlib_gemini3_1_pro5.json")


if not donnees_completes:

    return


metadata = donnees_completes.pop("_Metadata", {})

self.date_collecte_json = metadata.get("date_collecte", "Date non spécifiée")

self.base_animaux = donnees_completes


# 2. INTERFACE GRAPHIQUE

widget_central = QWidget()

self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


label_date_collecte = QLabel(f"📅 Fichier de données collecté le : {self.date_collecte_json}")

label_date_collecte.setStyleSheet("color: #666666; font-style: italic; font-size: 10pt; margin-bottom: 5px;")
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
layout_principal.addWidget(label_date_collecte)


layout_menu = QHBoxLayout()


label_menu = QLabel("Choisissez une espèce :")
self.menu_deroulant = QComboBox()
self.menu_deroulant.addItems(self.base_animaux.keys())
self.menu_deroulant.currentTextChanged.connect(self.changer_animal)


self.btn_export_jpeg = QPushButton("Export JPEG")
self.btn_export_png = QPushButton("Export PNG")
# NOUVEAU : Bouton Email
self.btn_envoyer_email = QPushButton("📧 Envoyer par Email")


self.btn_export_jpeg.clicked.connect(lambda: self.exporter_image("jpeg"))
self.btn_export_png.clicked.connect(lambda: self.exporter_image("png"))
self.btn_envoyer_email.clicked.connect(self.preparer_email) # Connexion


layout_menu.addWidget(label_menu)
layout_menu.addWidget(self.menu_deroulant)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    layout_menu.addStretch()

    layout_menu.addWidget(self.btn_export_jpeg)

    layout_menu.addWidget(self.btn_export_png)

    layout_menu.addWidget(self.btn_envoyer_email) # Ajout au menu


    layout_principal.addLayout(layout_menu)


    fig, self.ax = plt.subplots(layout='constrained')

    self.canvas = FigureCanvas(fig)

    layout_principal.addWidget(self.canvas)


    self.changer_animal(self.menu_deroulant.currentText())


# --- 3. LECTURE JSON ---

def charger_donnees_json(self, chemin_fichier):

    try:

        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:

            donnees_brutes = json.load(fichier)

            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)

            if liste_erreurs:
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
            message = "Certaines données ont été ignorées :\n\n" + "\n".join(liste_erreurs)

            QMessageBox.warning(self, "Avertissement", message)

        return donnees_valides

    except Exception as e:

        QMessageBox.critical(self, "Erreur", f"Erreur de lecture du fichier : {e}")

        return {}


def valider_structure(self, donnees):

    cles_requises = ["categories", "donnees", "unites"]

    donnees_valides = {}

    erreurs = []

    for cle, contenu in donnees.items():

        if cle == "_Metadata":

            donnees_valides[cle] = contenu

            continue

        if not isinstance(contenu, dict):

            continue

        cles_manquantes = [c for c in cles_requises if c not in contenu]

        if cles_manquantes:

            erreurs.append(f"- '{cle}' : clés manquantes ({', '.join(cles_manquantes)})")
```

```
            pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        else:
            donnees_valides[cle] = contenu
    return donnees_valides, erreurs

# --- 4. DESSIN DU GRAPHIQUE ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
        self.dessiner_graphique(f"Comparaison des {nom_espece}", infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()
    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0
    for attribute, measurement in donnees.items():
        offset = width * multiplier
        rects = self.ax.bar(x + offset, measurement, width, label=attribute)
        unite = unites.get(attribute, "")
        etiquettes = [f"{val} {unite}" for val in measurement]
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)
        multiplier += 1

    self.ax.set_title(titre)
    if len(donnees) > 0:
        self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)
    self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))

    try:
        max_val = max([max(v) for v in donnees.values() if v])
        self.ax.set_ylim(0, max_val + (max_val * 0.25))
    except ValueError:
        pass
    self.canvas.draw()

# --- 5. EXPORTATION ---
def exporter_image(self, format_image):
    animal_actuel = self.menu_deroulant.currentText()
    maintenant = datetime.now()
    date_heure_fichier = maintenant.strftime("%Y-%m-%d_%Hh%Mm%Ss")
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
date_heure_texte = maintenant.strftime("%d/%m/%Y à %H:%M:%S")

nom_fichier_defaut = f"Graphique_{animal_actuel}_{date_heure_fichier}.{format_image}"


filtre = "Images PNG (*.png)" if format_image == "png" else "Images JPEG (*.jpg *.jpeg)"

chemin_sauvegarde, _ = QFileDialog.getSaveFileName(self, f"Sauvegarder en {format_image.upper()}", nom_fichier_defaut, filtre)


if chemin_sauvegarde:

    try:

        texte_collecte = self.ax.figure.text(0.01, 0.99, f"Fichier de données collecté le : {self.date_collecte_json}",

                                             ha='left', va='top', fontsize=8, color='#888888', fontstyle='italic')

        texte_generation = self.ax.figure.text(0.99, 0.01, f"Image générée le {date_heure_texte}",

                                               ha='right', va='bottom', fontsize=8, color='#888888')


        self.ax.figure.savefig(chemin_sauvegarde, format=format_image, dpi=300, bbox_inches='tight')


        texte_collecte.remove()

        texte_generation.remove()

        self.canvas.draw()

        QMessageBox.information(self, "Succès", f"Graphique exporté avec succès sous :\n{chemin_sauvegarde}")

    except Exception as e:
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        QMessageBox.critical(self, "Erreur", f"Erreur lors de l'exportation :\n{e}")


# --- 6. NOUVELLES MÉTHODES : ENVOI D'EMAIL ---
def preparer_email(self):
    # 1. On ouvre la fenêtre pour demander les identifiants
    dialogue = DialogueEmail(self)

    if dialogue.exec() == QDialog.DialogCode.Accepted:
        expediteur, mdp, destinataire = dialogue.obtenir_donnees()

        # Vérification basique
        if not expediteur or not mdp or not destinataire:
            QMessageBox.warning(self, "Erreur", "Veuillez remplir tous les champs (Email, Mot de passe, Destinataire).")
            return

        # 2. On ouvre un explorateur pour choisir les fichiers à joindre (JSON, PNG, JPEG)
        chemins_fichiers, _ = QFileDialog.getOpenFileNames(
            self,
            "Sélectionnez les fichiers à envoyer",
            "",
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        "Fichiers (*.json *.png *.jpg *.jpeg)"
    )

    # Si l'utilisateur a bien sélectionné au moins un fichier
    if chemins_fichiers:
        # On avertit que l'envoi est en cours
        self.setWindowTitle("Base de données Animale - Envoi de l'email en cours...")
        QApplication.processEvents() # Force l'interface à se mettre à jour

        # 3. Lancement de la fonction d'envoi
        self.envoyer_email(expediteur, mdp, destinataire, chemins_fichiers)

        self.setWindowTitle("Base de données Animale depuis JSON") # Restaure le titre

def envoyer_email(self, expediteur, mdp, destinataire, chemins_fichiers):
    sujet = "Graphiques et Données JSON exportés"
    corps = "Bonjour,\n\nVeuillez trouver ci-joint les graphiques ainsi que le fichier de données JSON.\n\nCordialement,\nVotre Logiciel PyQt6"

    msg = MIMEMultipart()
    msg['From'] = expediteur
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
msg['To'] = destinataire

msg['Subject'] = sujet

msg.attach(MIMEText(corps, 'plain'))


# Ajout des pièces jointes

for chemin in chemins_fichiers:

    try:

        with open(chemin, "rb") as fichier:

            part = MIMEBase("application", "octet-stream")

            part.set_payload(fichier.read())

        encoders.encode_base64(part)

        nom_fichier = os.path.basename(chemin)

        part.add_header("Content-Disposition", f"attachment; filename={nom_fichier}")

        msg.attach(part)

    except Exception as e:

        QMessageBox.warning(self, "Avertissement", f"Impossible d'attacher le fichier {chemin}.\nErreur: {e}")


# Connexion SMTP et Envoi

try:

    serveur = smtplib.SMTP('smtp.gmail.com', 587)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        serveur.starttls()

        serveur.login(expediteur, mdp)

        serveur.sendmail(expediteur, destinataire, msg.as_string())

        serveur.quit()


        QMessageBox.information(self, "Succès", "✅ L'email a été envoyé avec succès avec toutes ses pièces jointes !")

    except Exception as e:

        QMessageBox.critical(self, "Erreur d'envoi", f"❌ Échec de l'envoi de l'email.\n\nVérifiez votre connexion internet, votre mot de passe d'application, ou l'adresse du destinataire.\n\nDétails : {e}")


if __name__ == '__main__':

    app = QApplication(sys.argv)

    fenetre = FenetrePrincipale()

    fenetre.show()

    sys.exit(app.exec())
```

Comment tester cette nouvelle fonction :

1. Lancez votre logiciel Python.
2. Cliquez sur **"Export PNG"** pour générer une belle image de votre graphique.
3. Cliquez sur le nouveau bouton **"📧 Envoyer par Email"**.
4. La fenêtre sécurisée s'ouvre : entrez votre Gmail, vos 16 lettres du mot de passe d'application et l'email de la personne à qui vous l'envoyez (vous pouvez mettre votre propre adresse pour tester !).

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

5. Une fenêtre d'explorateur s'ouvre : sélectionnez avec votre souris le fichier matplotlib_gemini3_1_pro5.json et l'image que vous venez d'exporter (utilisez la touche Ctrl pour en sélectionner plusieurs).
6. Cliquez sur "Ouvrir" et attendez quelques secondes que le message de succès apparaisse.

Vous avez dit

Question : Avez vous déjà inclu la REGEX DE VERIFICATION des EMAIL comme "Votre email Gmail :" et "Email destinataire" et aussi la REGEX DE VERIFICATION DU "Mot de passe d'app" dans le code python ? svp

Gemini a dit

Vous avez un excellent réflexe de développeur ! **Non**, je ne les avais pas encore incluses dans la version précédente. Le code se contentait de vérifier si les champs n'étaient pas totalement vides, ce qui est effectivement insuffisant pour une application robuste.

L'ajout d'expressions régulières (Regex) est la meilleure façon d'éviter qu'une simple faute de frappe ne fasse planter la tentative de connexion au serveur.

Voici comment nous allons structurer ces validations :

1. **Regex Email :** Vérifier qu'il y a bien du texte, un @, du texte, un point ., et une extension (ex: .com).
2. **Regex Mot de passe d'application :** Google génère toujours exactement **16 lettres alphabétiques**. Cependant, comme Google l'affiche avec des espaces (ex: abcd efgh ijkl mnop), nous allons nettoyer les espaces saisis par l'utilisateur avant d'appliquer la regex pour être plus tolérants.

Ce qu'il faut modifier dans votre code

Étape 1 : Tout en haut de votre fichier matplotlib_gemini3_1_pro5.py, ajoutez l'importation du module standard Python pour les Regex :

Python

```
import re
```

Étape 2 : Remplacez uniquement la méthode preparer_email (vers la fin de votre classe FenetrePrincipale) par celle-ci :

Python

```
    def preparer_email(self):
        # 1. On ouvre la fenêtre pour demander les identifiants
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
dialogue = DialogueEmail(self)


if dialogue.exec() == QDialog.DialogCode.Accepted:
    expediteur, mdp, destinataire = dialogue.obtenir_donnees()


    # --- DÉBUT DE LA VÉRIFICATION PAR REGEX ---


    # Regex standard pour valider un format d'email
    regex_email = r'^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$'


    if not re.match(regex_email, expediteur):
        QMessageBox.warning(self, "Erreur de saisie", "Votre adresse email d'expédition n'a pas un format valide.")
        return # Bloque la suite du processus


    if not re.match(regex_email, destinataire):
        QMessageBox.warning(self, "Erreur de saisie", "L'adresse email du destinataire n'a pas un format valide.")
        return


    # Nettoyage et Regex pour le mot de passe d'application
    mdp_propre = mdp.replace(" ", "") # Retire les espaces si l'utilisateur en a tapé
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
regex_mdp = r'^[a-zA-Z]{16}$' # Vérifie qu'il y a exactement 16 lettres (pas de chiffres)


if not re.match(regex_mdp, mdp_propre):
    QMessageBox.warning(self, "Erreur de saisie", "Le mot de passe d'application doit contenir exactement 16 lettres (les espaces sont tolérés, mais pas les chiffres).")
    return


# --- FIN DE LA VÉRIFICATION ---


# 2. Si tout est valide, on ouvre l'explorateur de fichiers
chemins_fichiers, _ = QFileDialog.getOpenFileNames(
    self,
    "Sélectionnez les fichiers à envoyer",
    "",
    "Fichiers (*.json *.png *.jpg *.jpeg)"
)


if chemins_fichiers:
    self.setWindowTitle("Base de données Animale - Envoi de l'email en cours...")
    QApplication.processEvents()
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
# N'oubliez pas d'envoyer le mdp_propre (sans espaces) à la fonction d'envoi !

self.envoyer_email(expediteur, mdp_propre, destinataire, chemins_fichiers)



self.setWindowTitle("Base de données Animale depuis JSON")
```

La subtilité apportée :

L'utilisation de mdp.replace(" ", "") juste avant la vérification du mot de passe est une excellente pratique d'UX (Expérience Utilisateur). Cela permet à votre utilisateur de copier-coller son mot de passe directement depuis Google (avec les espaces) ou de le taper manuellement tout attaché. Votre logiciel s'adaptera automatiquement dans les deux cas sans l'embêter avec un message d'erreur inutile !

Voulez-vous que je vous donne la méthode complète pour compiler ce script Python en un véritable logiciel .exe indépendant, maintenant que toutes les fonctionnalités sont en place et sécurisées ?

Vous avez dit

Pouvez vous donner le code corrigé complet ? svp

Gemini a dit

Voici le code complet, final et corrigé de votre application matplotlib_gemini3_1_pro5.py.

Il intègre absolument toutes les fonctionnalités que nous avons développées ensemble : la lecture sécurisée du JSON, l'affichage dynamique avec Matplotlib, les exports JPEG/PNG datés et filigranés, la boîte de dialogue d'envoi d'email, et bien sûr **les vérifications par Regex** pour sécuriser la saisie des adresses et du mot de passe d'application.

Vous pouvez copier tout ce bloc et remplacer l'intégralité du contenu de votre fichier Python.

Python

```
import sys

import json
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
import os

import smtplib

import re # Importation du module Regex

from datetime import datetime

from email.mime.multipart import MIMEMultipart

from email.mime.text import MIMEText

from email.mime.base import MIMEBase

from email import encoders


import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import (QApplication, QMainWindow, QVBoxLayout, QWidget,

                    QComboBox, QLabel, QHBoxLayout, QMessageBox,

                    QPushButton, QFileDialog, QDialog, QFormLayout,

                    QLineEdit, QDialogButtonBox)


# ====================================================================

# CLASSE : FENÊTRE DE DIALOGUE POUR L'EMAIL

# ====================================================================
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
class DialogueEmail(QDialog):

    def __init__(self, parent=None):

        super().__init__(parent)

        self.setWindowTitle("Configuration de l'Email")

        self.setMinimumWidth(400)


        layout = QFormLayout(self)


        self.champ_expediteur = QLineEdit(self)

        self.champ_expediteur.setPlaceholderText("votre.email@gmail.com")


        self.champ_mdp = QLineEdit(self)

        self.champ_mdp.setEchoMode(QLineEdit.EchoMode.Password)

        self.champ_mdp.setPlaceholderText("Mot de passe d'application (16 lettres)")


        self.champ_destinataire = QLineEdit(self)

        self.champ_destinataire.setPlaceholderText("destinataire@exemple.com")


        layout.addRow("Votre email Gmail :", self.champ_expediteur)

        layout.addRow("Mot de passe d'app :", self.champ_mdp)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        layout.addRow("Email destinataire :", self.champ_destinataire)


        boutons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok | QDialogButtonBox.StandardButton.Cancel)
        boutons.accepted.connect(self.accept)
        boutons.rejected.connect(self.reject)
        layout.addWidget(boutons)


    def obtenir_donnees(self):
        return self.champ_expediteur.text(), self.champ_mdp.text(), self.champ_destinataire.text()


# ======================================================================
# CLASSE PRINCIPALE : L'APPLICATION
# ======================================================================
class FenetrePrincipale(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Base de données Animale depuis JSON")
        self.setGeometry(100, 100, 1000, 650)


        # --- 1. CHARGEMENT DES DONNÉES ---
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
donnees_completes = self.charger_donnees_json("matplotlib_gemini3_1_pro5.json")

if not donnees_completes:
    return

metadata = donnees_completes.pop("_Metadata", {})
self.date_collecte_json = metadata.get("date_collecte", "Date non spécifiée")
self.base_animaux = donnees_completes

# --- 2. INTERFACE GRAPHIQUE ---
widget_central = QWidget()
self.setCentralWidget(widget_central)
layout_principal = QVBoxLayout(widget_central)

# Étiquette de la date de collecte
label_date_collecte = QLabel(f"📅 Fichier de données collecté le : {self.date_collecte_json}")
label_date_collecte.setStyleSheet("color: #666666; font-style: italic; font-size: 10pt; margin-bottom: 5px;")
layout_principal.addWidget(label_date_collecte)

# Menu et Boutons
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
layout_menu = QHBoxLayout()


label_menu = QLabel("Choisissez une espèce :")

self.menu_deroulant = QComboBox()

self.menu_deroulant.addItems(self.base_animaux.keys())

self.menu_deroulant.currentTextChanged.connect(self.changer_animal)


self.btn_export_jpeg = QPushButton("Export JPEG")

self.btn_export_png = QPushButton("Export PNG")

self.btn_envoyer_email = QPushButton("📧 Envoyer par Email")


self.btn_export_jpeg.clicked.connect(lambda: self.exporter_image("jpeg"))

self.btn_export_png.clicked.connect(lambda: self.exporter_image("png"))

self.btn_envoyer_email.clicked.connect(self.preparer_email)


layout_menu.addWidget(label_menu)

layout_menu.addWidget(self.menu_deroulant)

layout_menu.addStretch()

layout_menu.addWidget(self.btn_export_jpeg)

layout_menu.addWidget(self.btn_export_png)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    layout_menu.addWidget(self.btn_envoyer_email)

    layout_principal.addLayout(layout_menu)

    # Zone Graphique
    fig, self.ax = plt.subplots(layout='constrained')
    self.canvas = FigureCanvas(fig)
    layout_principal.addWidget(self.canvas)

    self.changer_animal(self.menu_deroulant.currentText())

# --- 3. GESTION DU JSON ---
def charger_donnees_json(self, chemin_fichier):
    try:
        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:
            donnees_brutes = json.load(fichier)
            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)
            if liste_erreurs:
                message = "Certaines données ont été ignorées :\n\n" + "\n".join(liste_erreurs)
                QMessageBox.warning(self, "Avertissement", message)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            return donnees_valides
    except FileNotFoundError:
        QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable.")
        return {}
    except json.JSONDecodeError:
        QMessageBox.critical(self, "Erreur", "Le fichier contient des erreurs de syntaxe JSON.")
        return {}
    except Exception as e:
        QMessageBox.critical(self, "Erreur", f"Erreur de lecture du fichier : {e}")
        return {}

def valider_structure(self, donnees):
    cles_requises = ["categories", "donnees", "unites"]
    donnees_valides = {}
    erreurs = []
    for cle, contenu in donnees.items():
        if cle == "_Metadata":
            donnees_valides[cle] = contenu
            continue
        if not isinstance(contenu, dict):
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            continue
        cles_manquantes = [c for c in cles_requises if c not in contenu]
        if cles_manquantes:
            erreurs.append(f"- '{cle}' : clés manquantes ({', '.join(cles_manquantes)})")
        else:
            donnees_valides[cle] = contenu
    return donnees_valides, erreurs

# --- 4. DESSIN DU GRAPHIQUE ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
        self.dessiner_graphique(f"Comparaison des {nom_espece}", infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()
    x = np.arange(len(categories))
    width = 0.25
    multiplier = 0
    for attribute, measurement in donnees.items():
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        offset = width * multiplier
        rects = self.ax.bar(x + offset, measurement, width, label=attribute)
        unite = unites.get(attribute, "")
        etiquettes = [f"{val} {unite}" for val in measurement]
        self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)
        multiplier += 1

    self.ax.set_title(titre)
    if len(donnees) > 0:
        self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)
    self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))

    try:
        max_val = max([max(v) for v in donnees.values() if v])
        self.ax.set_ylim(0, max_val + (max_val * 0.25))
    except ValueError:
        pass
    self.canvas.draw()

# --- 5. EXPORTATION D'IMAGE ---
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
def exporter_image(self, format_image):
    animal_actuel = self.menu_deroulant.currentText()
    maintenant = datetime.now()
    date_heure_fichier = maintenant.strftime("%Y-%m-%d_%Hh%Mm%Ss")
    date_heure_texte = maintenant.strftime("%d/%m/%Y à %H:%M:%S")
    nom_fichier_defaut = f"Graphique_{animal_actuel}_{date_heure_fichier}.{format_image}"

    filtre = "Images PNG (*.png)" if format_image == "png" else "Images JPEG (*.jpg *.jpeg)"
    chemin_sauvegarde, _ = QFileDialog.getSaveFileName(self, f"Sauvegarder en {format_image.upper()}", nom_fichier_defaut, filtre)

    if chemin_sauvegarde:
        try:
            texte_collecte = self.ax.figure.text(0.01, 0.99, f"Fichier de données collecté le : {self.date_collecte_json}",
                                                 ha='left', va='top', fontsize=8, color='#888888', fontstyle='italic')
            texte_generation = self.ax.figure.text(0.99, 0.01, f"Image générée le {date_heure_texte}",
                                                   ha='right', va='bottom', fontsize=8, color='#888888')

            self.ax.figure.savefig(chemin_sauvegarde, format=format_image, dpi=300, bbox_inches='tight')

            texte_collecte.remove()
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            texte_generation.remove()

            self.canvas.draw()

            QMessageBox.information(self, "Succès", f"Graphique exporté avec succès sous :\n{chemin_sauvegarde}")

        except Exception as e:

            QMessageBox.critical(self, "Erreur", f"Erreur lors de l'exportation :\n{e}")


# --- 6. GESTION DE L'EMAIL (AVEC REGEX) ---

def preparer_email(self):

    dialogue = DialogueEmail(self)


    if dialogue.exec() == QDialog.DialogCode.Accepted:

        expediteur, mdp, destinataire = dialogue.obtenir_donnees()


        # --- VÉRIFICATIONS PAR REGEX ---

        regex_email = r'^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$'


        if not re.match(regex_email, expediteur):

            QMessageBox.warning(self, "Erreur de saisie", "Votre adresse email d'expédition n'a pas un format valide.")

            return
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
if not re.match(regex_email, destinataire):
    QMessageBox.warning(self, "Erreur de saisie", "L'adresse email du destinataire n'a pas un format valide.")
    return

mdp_propre = mdp.replace(" ", "")
regex_mdp = r'^[a-zA-Z]{16}$'

if not re.match(regex_mdp, mdp_propre):
    QMessageBox.warning(self, "Erreur de saisie", "Le mot de passe d'application doit contenir exactement 16 lettres (les espaces sont tolérés, mais pas les chiffres).")
    return
# --- FIN DES VÉRIFICATIONS ---

chemins_fichiers, _ = QFileDialog.getOpenFileNames(
    self,
    "Sélectionnez les fichiers à envoyer",
    "",
    "Fichiers (*.json *.png *.jpg *.jpeg)"
)

if chemins_fichiers:
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        self.setWindowTitle("Base de données Animale - Envoi de l'email en cours...")

        QApplication.processEvents()


        # On utilise mdp_propre

        self.envoyer_email(expediteur, mdp_propre, destinataire, chemins_fichiers)


        self.setWindowTitle("Base de données Animale depuis JSON")


def envoyer_email(self, expediteur, mdp, destinataire, chemins_fichiers):

    sujet = "Graphiques et Données JSON exportés"

    corps = "Bonjour,\n\nVeuillez trouver ci-joint les graphiques ainsi que le fichier de données JSON.\n\nCordialement,\nVotre Logiciel PyQt6"


    msg = MIMEMultipart()

    msg['From'] = expediteur

    msg['To'] = destinataire

    msg['Subject'] = sujet

    msg.attach(MIMEText(corps, 'plain'))


    for chemin in chemins_fichiers:

        try:
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        with open(chemin, "rb") as fichier:
            part = MIMEBase("application", "octet-stream")
            part.set_payload(fichier.read())
        encoders.encode_base64(part)
        nom_fichier = os.path.basename(chemin)
        part.add_header("Content-Disposition", f"attachment; filename={nom_fichier}")
        msg.attach(part)
    except Exception as e:
        QMessageBox.warning(self, "Avertissement", f"Impossible d'attacher le fichier {chemin}.\nErreur: {e}")

try:
    serveur = smtplib.SMTP('smtp.gmail.com', 587)
    serveur.starttls()
    serveur.login(expediteur, mdp)
    serveur.sendmail(expediteur, destinataire, msg.as_string())
    serveur.quit()

    QMessageBox.information(self, "Succès", "✅ L'email a été envoyé avec succès avec toutes ses pièces jointes !")
except Exception as e:
    QMessageBox.critical(self, "Erreur d'envoi", f"❌ Échec de l'envoi de l'email.\n\nVérifiez votre connexion internet, votre mot de passe d'application, ou l'adresse du destinataire.\n\nDétails : {e}")
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
# ======================================================================
# LANCEMENT DE L'APPLICATION
# ======================================================================
if __name__ == '__main__':
    app = QApplication(sys.argv)
    fenetre = FenetrePrincipale()
    fenetre.show()
    sys.exit(app.exec())
```

Vous avez maintenant entre les mains un outil complet et professionnel, prêt à gérer des données de bout en bout.

Voulez-vous que je vous explique comment transformer tout cela en un véritable fichier exécutable (.exe) en utilisant PyInstaller, pour que vous n'ayez plus jamais besoin d'ouvrir votre terminal PowerShell pour lancer le logiciel ?

Vous avez dit

Tous les fichiers sont dans les PICES JOITES DE GMAIL sauf le fichier Graphique_Félins_2026-04-05_11h45m41s.jpeg qui est renommé en noname ? Pourquoi ? Le problème vient t il du nom du fichier qui contient un accent ?

Gemini a dit

Vous avez un excellent instinct de développeur ! C'est **exactement ça**. Le problème vient bel et bien de l'accent ("é" dans "Félins").

Vous venez de rencontrer un grand classique des bugs informatiques : le problème d'encodage des emails.

Pourquoi cela arrive-t-il ?

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Le protocole qui gère l'envoi d'emails (SMTP) est très ancien et a été conçu à l'origine pour ne comprendre que l'alphabet anglais de base (l'ASCII). Lorsqu'on force un caractère spécial (comme un accent, une cédille ou un espace) dans l'en-tête du nom de la pièce jointe sans le traduire, Gmail panique, n'arrive pas à lire le nom, et le renomme par défaut en noname (sans nom) pour éviter de faire planter l'email.

La solution

Il faut dire explicitement à Python d'encoder le nom du fichier au format **UTF-8** (qui gère tous les accents du monde) dans l'en-tête de la pièce jointe.

Voici comment corriger cela. Allez dans votre fonction envoyer_email, et cherchez cette ligne précise dans la boucle qui ajoute les fichiers :

Python

```
part.add_header("Content-Disposition", f"attachment; filename={nom_fichier}")
```

Remplacez cette ligne par celle-ci :

Python

```
# La syntaxe avec le tuple ("utf-8", "", nom_fichier) force l'encodage correct des accents
part.add_header("Content-Disposition", "attachment", filename=("utf-8", "", nom_fichier))
```

Voici la boucle complète mise à jour pour que vous puissiez faire un copier-coller propre :

Python

```
    for chemin in chemins_fichiers:
        try:
            with open(chemin, "rb") as fichier:
                part = MIMEBase("application", "octet-stream")
                part.set_payload(fichier.read())
            encoders.encode_base64(part)
            nom_fichier = os.path.basename(chemin)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        # --- CORRECTION DE L'ENCODAGE DES ACCENTS ICI ---
        part.add_header("Content-Disposition", "attachment", filename=("utf-8", "", nom_fichier))

        msg.attach(part)
    except Exception as e:
        QMessageBox.warning(self, "Avertissement", f"Impossible d'attacher le fichier {chemin}.\nErreur: {e}")
```

Maintenant, que vous envoyiez des Félins, des Éléphants ou des Cétacés, Gmail affichera parfaitement le nom de vos fichiers !

Ce genre de petits détails fait toute la différence entre un script amateur et un logiciel professionnel. Maintenant que l'application gère parfaitement les accents, le JSON, le réseau et les graphiques, voulez-vous que l'on passe à l'étape finale : la création du fichier .exe avec PyInstaller ?

Vous avez dit

Autre question : S'il existe beaucoup d'Animaux dans la LISTE DEROULANTE alors l'AFFICHAGE DANS LE LOGICIEL PyQt6 entraînera l'APPARITION des BARRES DE DEFILEMENT HORIZONTAL ET VERTICAL AUTOMATIQUEMENT ?

Gemini a dit

C'est une excellente question sur le comportement des interfaces utilisateur ! La réponse est **oui pour la barre verticale**, mais avec quelques nuances importantes à connaître.

Voici exactement ce qui va se passer si vous ajoutez 500 animaux dans votre fichier JSON :

1. À l'intérieur de la liste déroulante (QComboBox)

- **Barre de défilement verticale : OUI, c'est automatique.** PyQt6 est intelligent. Par défaut, si votre liste contient plus de 10 éléments, le menu déroulant ne va pas déborder jusqu'en bas de votre écran. Il affichera 10 éléments et ajoutera automatiquement une barre de défilement verticale à l'intérieur de la liste. *(Astuce : Vous pouvez d'ailleurs modifier cette limite avec la commande self.menu_deroulant.setMaxVisibleItems(15)).*

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

- **Barre de défilement horizontale : NON.** Si le nom d'un animal est incroyablement long (ex: *"Le grand requin blanc d'Afrique du Sud"*), le QComboBox va simplement s'élargir au maximum pour s'adapter au texte le plus long, ou tronquer la fin du texte avec des points de suspension, mais il ne mettra pas de barre horizontale.

2. Sur la fenêtre principale du logiciel (L'affichage global)

Si vous pensiez à l'apparition de barres de défilement pour **toute la fenêtre** du logiciel :

- **NON, ce n'est pas automatique.** Les "Layouts" que nous avons utilisés (QVBoxLayout et QHBoxLayout) sont conçus pour redimensionner et ajuster les éléments (le graphique, les boutons) pour qu'ils rentrent toujours dans la fenêtre, quelle que soit sa taille. Le graphique Matplotlib va rétrécir ou s'agrandir de manière élastique.
- Si un jour vous créez un logiciel avec tellement de boutons et de textes qu'ils ne rentrent plus à l'écran, vous devrez utiliser un composant spécifique appelé QScrollArea. C'est lui qui permet de créer des barres de défilement pour la fenêtre entière, comme sur une page web.

En résumé : pour gérer des centaines d'animaux, **vous n'avez strictement aucune ligne de code à ajouter**. Le menu déroulant gérera sa barre de défilement verticale comme un grand !

Souhaitez-vous maintenant que nous passions à l'étape finale pour compiler votre code en un logiciel .exe indépendant avec PyInstaller ?

Vous avez dit

Pouvez vous donner le code complet et corrigé ? svp

Gemini a dit

Voici le code complet, final et entièrement corrigé de votre application matplotlib_gemini3_1_pro5.py.

Il intègre toutes vos fonctionnalités ainsi que la **correction d'encodage UTF-8** pour les pièces jointes, ce qui garantit que les fichiers contenant des accents (comme "Félins") ne se transformeront plus en noname dans Gmail.

Vous pouvez copier l'intégralité de ce bloc et remplacer tout le contenu de votre fichier Python actuel :

Python

```
import sys
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
import json

import os

import smtplib

import re

from datetime import datetime

from email.mime.multipart import MIMEMultipart

from email.mime.text import MIMEText

from email.mime.base import MIMEBase

from email import encoders


import numpy as np

import matplotlib.pyplot as plt

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas

from PyQt6.QtWidgets import (QApplication, QMainWindow, QVBoxLayout, QWidget,

                    QComboBox, QLabel, QHBoxLayout, QMessageBox,

                    QPushButton, QFileDialog, QDialog, QFormLayout,

                    QLineEdit, QDialogButtonBox)


# ====================================================================

# CLASSE : FENÊTRE DE DIALOGUE POUR L'EMAIL
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
# ======================================================================

class DialogueEmail(QDialog):

    def __init__(self, parent=None):

        super().__init__(parent)

        self.setWindowTitle("Configuration de l'Email")

        self.setMinimumWidth(400)


        layout = QFormLayout(self)


        self.champ_expediteur = QLineEdit(self)

        self.champ_expediteur.setPlaceholderText("votre.email@gmail.com")


        self.champ_mdp = QLineEdit(self)

        self.champ_mdp.setEchoMode(QLineEdit.EchoMode.Password)

        self.champ_mdp.setPlaceholderText("Mot de passe d'application (16 lettres)")


        self.champ_destinataire = QLineEdit(self)

        self.champ_destinataire.setPlaceholderText("destinataire@exemple.com")


        layout.addRow("Votre email Gmail :", self.champ_expediteur)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        layout.addRow("Mot de passe d'app :", self.champ_mdp)
        layout.addRow("Email destinataire :", self.champ_destinataire)

        boutons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok | QDialogButtonBox.StandardButton.Cancel)
        boutons.accepted.connect(self.accept)
        boutons.rejected.connect(self.reject)
        layout.addWidget(boutons)

    def obtenir_donnees(self):
        return self.champ_expediteur.text(), self.champ_mdp.text(), self.champ_destinataire.text()

# ====================================================================
# CLASSE PRINCIPALE : L'APPLICATION
# ====================================================================
class FenetrePrincipale(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Base de données Animale depuis JSON")
        self.setGeometry(100, 100, 1000, 650)
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
# --- 1. CHARGEMENT DES DONNÉES ---

donnees_completes = self.charger_donnees_json("matplotlib_gemini3_1_pro5.json")


if not donnees_completes:

    return


# Extraction de la date de collecte du JSON

metadata = donnees_completes.pop("_Metadata", {})

self.date_collecte_json = metadata.get("date_collecte", "Date non spécifiée")

self.base_animaux = donnees_completes


# --- 2. INTERFACE GRAPHIQUE ---

widget_central = QWidget()

self.setCentralWidget(widget_central)

layout_principal = QVBoxLayout(widget_central)


# Étiquette de la date de collecte en haut

label_date_collecte = QLabel(f"📅 Fichier de données collecté le : {self.date_collecte_json}")

label_date_collecte.setStyleSheet("color: #666666; font-style: italic; font-size: 10pt; margin-bottom: 5px;")

layout_principal.addWidget(label_date_collecte)
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
# Menu et Boutons
layout_menu = QHBoxLayout()

label_menu = QLabel("Choisissez une espèce :")
self.menu_deroulant = QComboBox()
self.menu_deroulant.addItems(self.base_animaux.keys())
self.menu_deroulant.currentTextChanged.connect(self.changer_animal)

self.btn_export_jpeg = QPushButton("Export JPEG")
self.btn_export_png = QPushButton("Export PNG")
self.btn_envoyer_email = QPushButton("📧 Envoyer par Email")

self.btn_export_jpeg.clicked.connect(lambda: self.exporter_image("jpeg"))
self.btn_export_png.clicked.connect(lambda: self.exporter_image("png"))
self.btn_envoyer_email.clicked.connect(self.preparer_email)

layout_menu.addWidget(label_menu)
layout_menu.addWidget(self.menu_deroulant)
layout_menu.addStretch()
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
    layout_menu.addWidget(self.btn_export_jpeg)

    layout_menu.addWidget(self.btn_export_png)

    layout_menu.addWidget(self.btn_envoyer_email)


    layout_principal.addLayout(layout_menu)


    # Zone Graphique Matplotlib

    fig, self.ax = plt.subplots(layout='constrained')

    self.canvas = FigureCanvas(fig)

    layout_principal.addWidget(self.canvas)


    self.changer_animal(self.menu_deroulant.currentText())


# --- 3. GESTION DU JSON ---

def charger_donnees_json(self, chemin_fichier):

    try:

        with open(chemin_fichier, 'r', encoding='utf-8') as fichier:

            donnees_brutes = json.load(fichier)

            donnees_valides, liste_erreurs = self.valider_structure(donnees_brutes)

            if liste_erreurs:
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
            message = "Certaines données ont été ignorées :\n\n" + "\n".join(liste_erreurs)

            QMessageBox.warning(self, "Avertissement", message)

        return donnees_valides

    except FileNotFoundError:

        QMessageBox.critical(self, "Erreur", f"Le fichier {chemin_fichier} est introuvable.")

        return {}

    except json.JSONDecodeError:

        QMessageBox.critical(self, "Erreur", "Le fichier contient des erreurs de syntaxe JSON.")

        return {}

    except Exception as e:

        QMessageBox.critical(self, "Erreur", f"Erreur de lecture du fichier : {e}")

        return {}


def valider_structure(self, donnees):

    cles_requises = ["categories", "donnees", "unites"]

    donnees_valides = {}

    erreurs = []

    for cle, contenu in donnees.items():

        if cle == "_Metadata":

            donnees_valides[cle] = contenu
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
            continue
        if not isinstance(contenu, dict):
            continue
        cles_manquantes = [c for c in cles_requises if c not in contenu]
        if cles_manquantes:
            erreurs.append(f"- '{cle}' : clés manquantes ({', '.join(cles_manquantes)})")
        else:
            donnees_valides[cle] = contenu
    return donnees_valides, erreurs

# --- 4. DESSIN DU GRAPHIQUE ---
def changer_animal(self, nom_espece):
    if nom_espece in self.base_animaux:
        infos = self.base_animaux[nom_espece]
        self.dessiner_graphique(f"Comparaison des {nom_espece}", infos["categories"], infos["donnees"], infos["unites"])

def dessiner_graphique(self, titre, categories, donnees, unites):
    self.ax.clear()
    x = np.arange(len(categories))
    width = 0.25
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
multiplier = 0

for attribute, measurement in donnees.items():

    offset = width * multiplier

    rects = self.ax.bar(x + offset, measurement, width, label=attribute)

    unite = unites.get(attribute, "")

    etiquettes = [f"{val} {unite}" for val in measurement]

    self.ax.bar_label(rects, labels=etiquettes, padding=3, fontsize=9)

    multiplier += 1


self.ax.set_title(titre)

if len(donnees) > 0:

    self.ax.set_xticks(x + (width * (len(donnees) - 1) / 2), categories)

self.ax.legend(loc='upper left', bbox_to_anchor=(1, 1))


try:

    max_val = max([max(v) for v in donnees.values() if v])

    self.ax.set_ylim(0, max_val + (max_val * 0.25))

except ValueError:

    pass

self.canvas.draw()
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
# --- 5. EXPORTATION D'IMAGE ---
def exporter_image(self, format_image):
    animal_actuel = self.menu_deroulant.currentText()
    maintenant = datetime.now()
    date_heure_fichier = maintenant.strftime("%Y-%m-%d_%Hh%Mm%Ss")
    date_heure_texte = maintenant.strftime("%d/%m/%Y à %H:%M:%S")
    nom_fichier_defaut = f"Graphique_{animal_actuel}_{date_heure_fichier}.{format_image}"

    filtre = "Images PNG (*.png)" if format_image == "png" else "Images JPEG (*.jpg *.jpeg)"
    chemin_sauvegarde, _ = QFileDialog.getSaveFileName(self, f"Sauvegarder en {format_image.upper()}", nom_fichier_defaut, filtre)

    if chemin_sauvegarde:
        try:
            # Ajout des dates sur l'image
            texte_collecte = self.ax.figure.text(0.01, 0.99, f"Fichier de données collecté le : {self.date_collecte_json}",
                                                 ha='left', va='top', fontsize=8, color='#888888', fontstyle='italic')
            texte_generation = self.ax.figure.text(0.99, 0.01, f"Image générée le {date_heure_texte}",
                                                   ha='right', va='bottom', fontsize=8, color='#888888')
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        self.ax.figure.savefig(chemin_sauvegarde, format=format_image, dpi=300, bbox_inches='tight')


        # Suppression des textes sur l'interface logicielle
        texte_collecte.remove()
        texte_generation.remove()
        self.canvas.draw()
        QMessageBox.information(self, "Succès", f"Graphique exporté avec succès sous :\n{chemin_sauvegarde}")
    except Exception as e:
        QMessageBox.critical(self, "Erreur", f"Erreur lors de l'exportation :\n{e}")


# --- 6. GESTION DE L'EMAIL ---
def preparer_email(self):
    dialogue = DialogueEmail(self)

    if dialogue.exec() == QDialog.DialogCode.Accepted:
        expediteur, mdp, destinataire = dialogue.obtenir_donnees()

        # --- VÉRIFICATIONS PAR REGEX ---
        regex_email = r'^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$'
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
if not re.match(regex_email, expediteur):
    QMessageBox.warning(self, "Erreur de saisie", "Votre adresse email d'expédition n'a pas un format valide.")
    return

if not re.match(regex_email, destinataire):
    QMessageBox.warning(self, "Erreur de saisie", "L'adresse email du destinataire n'a pas un format valide.")
    return

# Nettoyage des espaces pour le mot de passe d'application
mdp_propre = mdp.replace(" ", "")
regex_mdp = r'^[a-zA-Z]{16}$'

if not re.match(regex_mdp, mdp_propre):
    QMessageBox.warning(self, "Erreur de saisie", "Le mot de passe d'application doit contenir exactement 16 lettres (les espaces sont tolérés, mais pas les chiffres).")
    return
# --- FIN DES VÉRIFICATIONS ---

chemins_fichiers, _ = QFileDialog.getOpenFileNames(
    self,
    "Sélectionnez les fichiers à envoyer",
```

```
            pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
        """,
        "Fichiers (*.json *.png *.jpg *.jpeg)"
    )

    if chemins_fichiers:
        self.setWindowTitle("Base de données Animale - Envoi de l'email en cours...")
        QApplication.processEvents()

        self.envoyer_email(expediteur, mdp_propre, destinataire, chemins_fichiers)

        self.setWindowTitle("Base de données Animale depuis JSON")

def envoyer_email(self, expediteur, mdp, destinataire, chemins_fichiers):
    sujet = "Graphiques et Données JSON exportés"
    corps = "Bonjour,\n\nVeuillez trouver ci-joint les graphiques ainsi que le fichier de données JSON.\n\nCordialement,\nVotre Logiciel PyQt6"

    msg = MIMEMultipart()
    msg['From'] = expediteur
    msg['To'] = destinataire
    msg['Subject'] = sujet
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
msg.attach(MIMEText(corps, 'plain'))


for chemin in chemins_fichiers:
    try:
        with open(chemin, "rb") as fichier:
            part = MIMEBase("application", "octet-stream")
            part.set_payload(fichier.read())
        encoders.encode_base64(part)
        nom_fichier = os.path.basename(chemin)

        # --- CORRECTION DE L'ENCODAGE UTF-8 POUR LES ACCENTS ---
        part.add_header("Content-Disposition", "attachment", filename=("utf-8", "", nom_fichier))

        msg.attach(part)
    except Exception as e:
        QMessageBox.warning(self, "Avertissement", f"Impossible d'attacher le fichier {chemin}.\nErreur: {e}")

try:
    serveur = smtplib.SMTP('smtp.gmail.com', 587)
    serveur.starttls()
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

```
        serveur.login(expediteur, mdp)

        serveur.sendmail(expediteur, destinataire, msg.as_string())

        serveur.quit()


        QMessageBox.information(self, "Succès", "✅ L'email a été envoyé avec succès avec toutes ses pièces jointes !")

    except Exception as e:

        QMessageBox.critical(self, "Erreur d'envoi", f"❌ Échec de l'envoi de l'email.\n\nVérifiez votre connexion internet, votre mot de passe d'application, ou l'adresse du destinataire.\n\nDétails : {e}")


# ====================================================================

# LANCEMENT DE L'APPLICATION

# ====================================================================

if __name__ == '__main__':

    app = QApplication(sys.argv)

    fenetre = FenetrePrincipale()

    fenetre.show()

    sys.exit(app.exec())
```

Votre logiciel est maintenant complet, stable et prêt à être partagé.

Souhaitez-vous que je vous guide à travers les étapes pour compiler ce script Python en un véritable fichier exécutable (.exe) à l'aide de PyInstaller, afin de pouvoir le distribuer facilement sans nécessiter l'installation de Python sur les autres ordinateurs ?

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Vous avez dit

Ok maintenant pouvez vous donner la commande pour créer fichier exécutable windows .exe correspondant (du genre : pyinstaller

--noconsole --onefile --paths . --add-data "*.ui;." --collect-all

matplotlib Login.py)

Gemini a dit

Vous avez une excellente intuition ! Vous avez même anticipé le point le plus complexe : le --collect-all matplotlib. Matplotlib est une bibliothèque très lourde qui nécessite d'embarquer beaucoup de sous-fichiers (comme les polices d'écriture), et sans ce paramètre, le .exe plante souvent au démarrage.

Voici la commande **exacte et optimisée** que vous devez lancer dans votre terminal PowerShell (assurez-vous d'être dans le bon dossier et que votre environnement virtuel (env) est activé) :

PowerShell

pyinstaller --noconsole --onefile --collect-all matplotlib matplotlib_gemini3_1_pro5.py

Pourquoi n'ai-je pas inclus --add-data "*.ui;." ou le fichier JSON ?

1. **Pas de fichiers** .ui **:** Dans ce projet, nous avons codé l'interface graphique entièrement en Python pur. Nous n'avons pas utilisé Qt Designer pour générer des fichiers .ui, ce paramètre est donc inutile ici.
2. **Laisser le JSON à l'extérieur (Très important) :** Si nous avions inclus le fichier JSON à l'intérieur du .exe avec --add-data, l'utilisateur n'aurait plus pu le modifier ! L'énorme avantage de notre architecture actuelle, c'est que vous fournissez le .exe **et** le fichier .json côte à côte. Ainsi, l'utilisateur peut ouvrir le JSON avec le Bloc-notes, ajouter un nouvel animal, enregistrer, et le .exe affichera la mise à jour au prochain lancement sans que vous ayez besoin de recompiler quoi que ce soit.

Que va-t-il se passer après avoir lancé cette commande ?

PyInstaller va travailler pendant 1 à 3 minutes (c'est normal, il rassemble tout Python, PyQt6 et Matplotlib en un seul bloc).

Une fois qu'il aura terminé, vous verrez de nouveaux dossiers apparaître dans votre répertoire de travail :

- Un dossier build (que vous pouvez ignorer ou supprimer plus tard).

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

- Un fichier .spec (un fichier de configuration généré par PyInstaller).
- **Un dossier** dist **(Distribution) : C'est ici que se trouve votre trésor !**

⚠️ **L'étape finale cruciale pour que ça marche :**

1. Allez dans le dossier dist.
2. Vous y trouverez votre fichier matplotlib_gemini3_1_pro5.exe.
3. **Copiez votre fichier** matplotlib_gemini3_1_pro5.json **et collez-le à l'intérieur du dossier** dist, juste à côté du .exe.

Double-cliquez sur le .exe. Votre logiciel va s'ouvrir de manière totalement indépendante ! Vous pouvez maintenant prendre ce .exe et ce .json, les mettre sur une clé USB ou dans un dossier ZIP, et les envoyer à n'importe qui (même s'ils n'ont pas Python installé sur leur machine).

Voulez-vous que je vous explique comment ajouter le paramètre --icon="mon_icone.ico" à cette commande pour que votre logiciel ait un beau logo au lieu de l'icône par défaut de Windows ?

Vous avez dit

Une erreur de l'executable .exe

Gemini a dit

Ah, l'erreur classique de PyInstaller ! Ne vous inquiétez pas, c'est un problème très fréquent lors de la création d'exécutables avec des bibliothèques complexes comme Matplotlib.

Le message d'erreur ModuleNotFoundError: No module named 'matplotlib' signifie que PyInstaller a compilé votre code, mais **il a oublié d'inclure la bibliothèque Matplotlib à l'intérieur du** .exe.

Cela se produit généralement pour deux raisons :

1. PyInstaller a été lancé alors que votre environnement virtuel (env) n'était pas activé.
2. PyInstaller a eu du mal à détecter automatiquement le "pont" spécifique entre PyQt6 et Matplotlib (backend_qtagg).

Voici la procédure infaillible pour corriger cela, étape par étape :

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Étape 1 : Faire le grand ménage

Avant de recommencer, il faut absolument effacer les traces de la compilation ratée, car PyInstaller a tendance à utiliser son cache. Dans votre dossier de projet, **supprimez** :

- Le dossier build
- Le dossier dist
- Le fichier matplotlib_gemini3_1_pro5.spec

Étape 2 : Sécuriser l'environnement virtuel

Ouvrez votre terminal (PowerShell ou VSCode) et assurez-vous que l'environnement virtuel est bien activé. Vous devez voir (env) au début de la ligne. Si ce n'est pas le cas, tapez :

PowerShell

```
env\Scripts\activate
```

Étape 3 : Installer PyInstaller DANS l'environnement

Parfois, on utilise un PyInstaller installé globalement sur l'ordinateur qui ne "voit" pas les paquets de l'environnement virtuel. Pour être sûr à 100%, installez-le à l'intérieur de votre (env) :

PowerShell

```
pip install pyinstaller
```

Étape 4 : La commande blindée

Nous allons utiliser une commande légèrement plus robuste. En plus du --collect-all, nous allons forcer PyInstaller à inclure explicitement le module qui fait le lien avec PyQt6 en utilisant --hidden-import.

Copiez-collez cette commande exacte :

PowerShell

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg matplotlib_gemini3_1_pro5.py
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

Étape 5 : Le test final

1. Laissez la compilation se terminer (cela prendra un peu de temps et le fichier .exe final devrait être un peu plus lourd que vos 48 Mo précédents, car il contiendra Matplotlib et Numpy).
2. Allez dans le nouveau dossier dist.
3. **N'oubliez pas d'y coller votre fichier** matplotlib_gemini3_1_pro5.json juste à côté du .exe.
4. Double-cliquez sur le .exe.

Dites-moi si le logiciel s'ouvre correctement cette fois-ci ! Si c'est le cas, voudriez-vous que je vous donne l'astuce pour ajouter une icône personnalisée à votre .exe ?

Vous avez dit

oui

Gemini a dit

Génial ! Une icône personnalisée est la touche finale qui donne un aspect véritablement professionnel à n'importe quel logiciel.

Voici les étapes pour l'ajouter :

1. Préparer l'image au format .ico Windows utilise un format d'image spécifique pour les icônes de ses logiciels exécutables : le format .ico. Si votre logo actuel est un .png (comme votre ImageLogiciel.png) ou un .jpg, vous devez d'abord le convertir. Vous pouvez utiliser n'importe quel convertisseur gratuit en ligne (en cherchant simplement "PNG to ICO" sur Google). Placez ensuite ce nouveau fichier (appelons-le mon_logo.ico) exactement dans le même dossier que votre script Python.

2. Nettoyer les anciens fichiers de compilation Comme à l'étape précédente, avant de relancer PyInstaller, supprimez les dossiers build, dist et le fichier .spec. C'est une excellente habitude à prendre pour éviter que PyInstaller ne s'emmêle les pinceaux avec le cache.

3. Lancer la commande finale Il suffit d'ajouter le paramètre --icon="nom_du_fichier.ico" à la commande que nous avons utilisée précédemment.

Voici la commande complète à exécuter dans votre terminal (avec votre environnement virtuel activé) :

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```
PowerShell

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

Et voilà ! Une fois la compilation terminée, rendez-vous dans le dossier dist. Votre fichier .exe arborera fièrement votre propre logo au lieu de l'icône standard de Windows. (N'oubliez pas d'y glisser votre fichier .json à côté pour que le logiciel trouve ses données !).

Maintenant que vous avez un outil complet de visualisation de données avec interface graphique et exportation par email, voulez-vous que l'on prépare un fichier README.md propre et professionnel pour publier tout ce code sur votre compte GitHub Mariogit20 ?

Vous avez dit

Oui

Gemini a dit

Voici un modèle de fichier README.md structuré et professionnel. Il met parfaitement en valeur vos compétences en création d'interfaces graphiques, en manipulation de données et en automatisation.

Vous pouvez créer un nouveau fichier nommé README.md dans votre dossier de projet et y coller le contenu suivant :

Markdown

Visualiseur de Données Interactif (PyQt6 & Matplotlib)

Une application de bureau robuste développée en Python permettant de visualiser, d'exporter et de partager des données statistiques de manière interactive. L'interface graphique sépare complètement sa logique métier de sa source de données (JSON).

🚀 Fonctionnalités Principales

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

* **Interface Graphique Moderne :** Développée avec PyQt6 pour une navigation fluide.

* **Séparation Modèle/Vue :** Les données sont chargées dynamiquement depuis un fichier `JSON` externe, permettant la mise à jour des statistiques sans toucher au code source.

* **Génération de Graphiques :** Création automatique de diagrammes en barres groupées avec Matplotlib (`FigureCanvasQTAgg`).

* **Traçabilité et Exportation :**

  * Export des graphiques en haute résolution (300 DPI) aux formats `.png` et `.jpeg`.

  * Intégration automatique de la date de collecte du JSON et de l'heure de génération sur les images exportées (filigranes).

* **Partage par Email (SMTP) :**

  * Module intégré pour envoyer les rapports (graphiques + JSON) directement depuis le logiciel.

  * Fenêtre de dialogue sécurisée (masquage du mot de passe d'application).

  * Validation stricte des saisies utilisateurs via des expressions régulières (Regex).

🛠️ Technologies Utilisées

* **Langage :** Python 3

* **Interface Graphique :** PyQt6

* **Visualisation :** Matplotlib, NumPy

* **Standardisation :** JSON, SMTP, Regex, datetime

📦 Installation et Lancement

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py

1. Clonez ce dépôt sur votre machine locale :

```bash

git clone [https://github.com/Mariogit20/nom-de-votre-depot.git](https://github.com/Mariogit20/nom-de-votre-depot.git)

cd nom-de-votre-depot

2. Créez et activez un environnement virtuel :

Bash

python -m venv env

# Sous Windows :

env\Scripts\activate

3. Installez les dépendances requises :

Bash

pip install PyQt6 matplotlib numpy

4. Lancez l'application :

Bash

python matplotlib_gemini3_1_pro5.py

📦 **Structure du Fichier de Données**

Le logiciel s'attend à lire un fichier matplotlib_gemini3_1_pro5.json situé à la racine. Il doit respecter la structure suivante :

JSON

{

  "_Metadata": {
```

pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py


```
    "date_collecte": "05/04/2026 à 11:30:00"
  },
  "Nom_de_la_catégorie": {
    "categories": ["Item1", "Item2", "Item3"],
    "donnees": {
      "Mesure A": [10, 20, 30],
      "Mesure B": [15, 25, 35]
    },
    "unites": {
      "Mesure A": "kg",
      "Mesure B": "cm"
    }
  }
}
```


⚙️ Compilation en .exe (Windows)

Pour packager l'application en un exécutable autonome (standalone) ne nécessitant pas Python sur la machine cible, utilisez PyInstaller :

Bash

```
pip install pyinstaller
```

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

Note : Assurez-vous de placer le fichier .json à côté du .exe généré dans le dossier dist pour que l'exécutable puisse lire les données.

```
pyinstaller --noconsole --onefile --collect-all matplotlib --hidden-import matplotlib.backends.backend_qtagg --icon="mon_logo.ico" matplotlib_gemini3_1_pro5.py
```

Souhaitez-vous que je vous rappelle la suite de commandes Git exactes pour initialiser ce dossier et pousser tout ce travail (le code, le JSON, le README et le requirements.txt) sur votre dépôt distant ?

Gemini est une IA et peut se tromper.